Bangladesh University of Business and Technology (BUBT)

Department of Computer Science and Engineering

ElectroFix

Used Device Buy, Sell & Repair Platform

A Project Report

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer Science and Engineering

Submitted by:

Asif Raihan Rafi (22235103281)

Sadia Akter Nur (22235103286)

Samia Sultana (22235103292)

Raisha Zaman (22235103295)

Rakibul Islam (22235103299)

Supervised by:

Rubaiya Reza Sohana (RRSA)

Lecturer, Dept. of CSE

Bangladesh University of Business and Technology (BUBT)

Date: November 21, 2025

Certificate

This is to certify that the project titled **"ElectroFix: Used Device Buy, Sell & Repair Platform"** has been carried out by:

- Asif Raihan Rafi (22235103281)
- Sadia Akter Nur (22235103286)
- Samia Sultana (22235103292)
- Raisha Zaman (22235103295)
- Rakibul Islam (22235103299)

under my supervision in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering at the Department of Computer Science and Engineering, Bangladesh University of Business and Technology (BUBT).

Supervisor:

Rubaiya Reza Sohana (RRSA)

Lecturer, Dept. of CSE, BUBT

Acknowledgment

All praise goes to Almighty Allah for providing us with the strength, patience, and opportunity to complete this project successfully.

We express our sincere gratitude to our supervisor, **Rubaiya Reza Sohana (RRSA)**, Lecturer, Dept. of CSE, BUBT, for her continuous guidance, valuable feedback, and constant support throughout the project.

We would also like to thank the faculty members of the Department of Computer Science and Engineering for providing essential knowledge and resources.

Finally, we convey our appreciation to our families and friends for their encouragement, as well as our team members for their dedication, cooperation, and teamwork throughout the development of this project.

Abstract

The significant increase in the usage of electronic devices has created a growing demand for sustainable, reliable, and affordable solutions for buying, selling, and repairing used devices. ElectroFix is a web-based platform designed to facilitate secure device resale, repair service booking, automated diagnosis, and price prediction using AI-enabled modules.

This project implements a complete ecosystem featuring user authentication, device listing, repair service requests, an intelligent price prediction model, and a fault detection model based on text and image inputs. The system architecture follows a three-layered structure, integrating Frontend (HTML/CSS/JS), Backend (Python Django), and MongoDB database.

The platform supports a modular design, strict coding standards, comprehensive testing, and performance evaluation. This report presents the system requirements, analysis, design, implementation, testing strategies, results, and future enhancements for scaling ElectroFix into a fully functional production-grade platform.

Contents

1 Introduction **11**

1.1 Background 11

1.2 Problem Statement 12

1.3 Objectives 13

1.4 Scope of the Project 14

1.5 Significance of the Project 16

1.6 Motivation 17

1.7 Organization of the Report 17

2 Literature Review **19**

2.1 Used Device Marketplaces and Recommerce Platforms 19

2.1.1 Traditional Used Device Marketplaces 19

2.1.2 Modern Recommerce Platforms 20

2.2 Repair Service Management Systems 21

2.2.1 Offline Repair Ecosystems 21

2.2.2 Online Repair Platforms 21

2.3 Machine Learning for Price Prediction 22

2.4 Fault Detection Systems 23

2.4.1 Text-Based Fault Detection 23

2.4.2 Image-Based Fault Classification 23

2.5 E-Waste and Circular Economy Research 24

2.6 Technological Frameworks in Literature 24

2.6.1 Frontend Technologies 24

2.6.2 Backend Technologies 25

2.6.3 Database Models 25

2.7 Existing Gaps Identified in Literature 25

2.8 Summary 26

3 System Analysis **27**

3.1 Overview of the System 27

3.2 System Requirements Specification (SRS) 28

3.2.1 Functional Requirements 28

3.2.2 Non-Functional Requirements (NFR) 30

3.3 User Analysis 31

3.3.1 1. Customers 31

3.3.2 2. Technicians 31

3.3.3 3. Administrators 31

3.4 Feasibility Study 31

3.4.1 Technical Feasibility 32

3.4.2 Economic Feasibility 32

3.4.3 Operational Feasibility 32

3.5 Work Breakdown Structure (WBS) 32

3.5.1 WBS Level 1 32

3.5.2 WBS Level 2 Breakdown 33

3.6 Use Case Analysis 34

3.6.1 Use Case 1: User Login 34

3.6.2 Use Case 2: Device Listing 35

3.6.3 Use Case 3: Repair Request 35

3.7 Requirement Traceability Matrix (RTM) 35

3.8 Summary 36

3.9 System Architecture 37

3.10 Use Case Diagram 39

3.11 Class Diagram 41

3.12 Sequence Diagram 43

3.13 Activity Diagram 45

3.13.1 DFD Level 0 46

3.14 ER Diagram 47

4 Implementation **49**

4.1 Introduction . . . 49
4.2 Tools and Technologies Used . . . 50
4.2.1 Frontend Technologies . . . 50
4.2.2 Backend Technologies . . . 50
4.2.3 Database Technologies . . . 50
4.2.4 AI/ML Tools . . . 50
4.2.5 Development Tools . . . 51
4.3 System Architecture Implementation . . . 51
4.4 Directory Structure . . . 51
4.5 Frontend Implementation . . . 52
4.5.1 Homepage Implementation . . . 52
4.5.2 Sample Code: Navigation Bar . . . 53
4.5.3 Form Validation Implementation . . . 53
4.6 Backend Implementation . . . 53
4.6.1 Initialization . . . 53
4.6.2 JWT Authentication . . . 54
4.7 API Implementation . . . 55
4.7.1 User Registration API . . . 55
4.7.2 Price Prediction API . . . 55
4.8 Database Implementation . . . 55
4.8.1 Sample Document Structure . . . 56
4.9 Coding Standards Compliance . . . 56
4.10 Deployment . . . 56
4.10.1 Deployment Steps . . . 56
4.10.2 Environment Variables . . . 57
4.11 Summary . . . 57

5 Testing **58**
5.1 Introduction to Testing . . . 58
5.2 Testing Methodologies . . . 59
5.2.1 Black-Box Testing . . . 59
5.2.2 White-Box Testing . . . 59
5.2.3 Manual Testing . . . 59

5.2.4 Automated Testing . . . 60
5.3 Test Environment Setup . . . 60
5.4 UI/UX Testing . . . 60
5.4.1 UI/UX Test Cases . . . 61
5.5 Frontend Functional Testing . . . 61
5.5.1 Frontend Test Cases . . . 61
5.6 Backend API Testing . . . 62
5.7 Backend Testing . . . 62
5.8 Database Testing . . . 63
5.8.1 Database Test Cases . . . 63
5.9 Security Testing . . . 64
5.9.1 Security Test Cases . . . 64
5.10 Performance Testing . . . 64
5.11 AI Model Testing . . . 65
5.11.1 Price Prediction Model Test Cases . . . 65
5.11.2 Fault Detection Model Test Cases . . . 65
5.12 Integration Testing . . . 65
5.13 Deployment Testing . . . 66
5.14 Maintenance Testing . . . 66
5.15 Summary . . . 66

6 Results and Discussion 67

6.1 Introduction . . . 67
6.2 Functional Results . . . 68
6.3 UI/UX Evaluation Results . . . 69
6.3.1 Usability Score . . . 69
6.3.2 Interpretation . . . 69
6.4 System Performance Results . . . 70
6.4.1 API Response Times . . . 70
6.4.2 Interpretation . . . 70
6.5 Database Performance . . . 71
6.5.1 Results . . . 71
6.6 AI Model Results . . . 71

6.6.1 Price Prediction Model Evaluation . . . 71
6.6.2 Model Performance . . . 72
6.6.3 Interpretation . . . 72
6.6.4 Fault Detection Model Evaluation . . . 72
6.6.5 Confusion Matrix (Interpretive Description) . . . 72
6.6.6 Interpretation . . . 73
6.7 Security Test Results . . . 73
6.8 Integration Results . . . 73
6.9 User Acceptance Test (UAT) Results . . . 74
6.10 Discussion . . . 74
6.10.1 Strengths of ElectroFix . . . 74
6.10.2 Limitations . . . 74
6.10.3 Real-World Applicability . . . 75
6.11 Summary . . . 75

7 Conclusion and Future Work **76**
7.1 Conclusion . . . 76
7.2 Achievements of the System . . . 77
7.2.1 Technical Achievements . . . 78
7.2.2 AI Achievements . . . 78
7.2.3 User-centric Achievements . . . 78
7.3 Limitations . . . 78
7.4 Future Work . . . 79
7.4.1 1. Enhanced AI Models . . . 79
7.4.2 2. Integration of Marketplace Features . . . 79
7.4.3 3. Advanced Repair Management . . . 80
7.4.4 4. Mobile Application Development . . . 80
7.4.5 5. Multilingual and Localization Support . . . 80
7.4.6 6. Improved Security Infrastructure . . . 80
7.4.7 7. Integration with External APIs . . . 81
7.5 Final Remarks . . . 81

Appendices **82**

Appendix A: Software Requirements Specification (SRS) **83**
A.1 Introduction . . . 83
A.1.1 Purpose . . . 83
A.1.2 Scope . . . 83
A.1.3 Definitions, Acronyms, and Abbreviations . . . 84
A.2 Overall Description . . . 84
A.2.1 Product Perspective . . . 84
A.2.2 User Classes and Characteristics . . . 84
A.3 Functional Requirements . . . 84
A.3.1 User Registration and Login . . . 84
A.3.2 Device Buy/Sell Module . . . 85
A.3.3 Repair Booking System . . . 85
A.4 Non-Functional Requirements . . . 85
A.4.1 Performance . . . 85
A.4.2 Security . . . 85
A.4.3 Usability . . . 85
A.5 System Models . . . 86
A.6 Software Design Specification (SDS) . . . 87
A.7 Work Breakdown Structure (WBS) . . . 88
A.8 Work Breakdown Structure (WBS) . . . 88
A.9 Test Case Template . . . 92
A.10 Coding Standards Document . . . 94
A.11 Database Schema and Collections . . . 95
A.12 AI Model Training Dataset Samples . . . 97
A.13 System Screenshots . . . 98
A.14 Application Screenshots . . . 98

List of Figures

3.1 System Architecture Diagram of ElectroFix . . . 38

3.2 Use Case Diagram of ElectroFix 39
3.3 Class Diagram of ElectroFix 41
3.4 Sequence Diagram for Device Purchase Flow 43
3.5 Activity Diagram for Repair Request Processing 45
3.6 DFD Level 0 of ElectroFix 46
3.7 ER Diagram of ElectroFix 47

1 SRS Use Case Diagram 86
2 ElectroFix Work Breakdown Structure 89
3 Gantt Chart for ElectroFix Work Breakdown Structure 90
4 ElectroFix Homepage Interface 99
5 Login Page Interface 100
6 Signup Page Interface 101
7 User Dashboard Interface 102
8 Sell Device Form Interface 103
9 Repair Booking Page Interface 104
10 Repair Booking Form Interface 105
11 AI chat Bot Interface 106

List of Tables

3.1 Requirement Traceability Matrix 35

6.1 Summary of Functional Results 68
6.2 UI/UX Evaluation Results 69
6.3 API Response Time Results 70
6.4 Price Prediction Model Performance 72

Chapter 1

Introduction

1.1 Background

Over the last decade, global dependence on consumer electronics such as smartphones, laptops, tablets, and wearable devices has increased dramatically. The rapid advancement of technology has shortened the lifespan of electronic products, resulting in a continuous cycle of upgrades, replacements, and disposals. As a result, there has been a significant rise in the volume of used electronic devices entering secondary markets and repair ecosystems.

In developing countries such as Bangladesh, the demand for affordable second-hand devices is rising faster than ever. A considerable portion of the population prefers used devices due to their lower cost, accessibility, and practicality. Alongside device purchase, repair services remain essential because users frequently experience issues such as battery degradation, broken screens, charging faults, and software malfunction.

Despite high demand, the market suffers from several limitations: unreliable sellers, lack of transparency in pricing, unverified repairs, limited trust between buyers and sellers, and absence of streamlined communication channels. Customers often rely on unregulated repair shops with inconsistent service quality and no guarantee of authenticity. Additionally, most buying and selling of used devices take place through informal means such as social media groups, street markets, or personal referrals—leading to fraud, hidden defects, and safety concerns.

To address these challenges, the concept of a centralized, trustworthy, and technology-supported platform has become vital. ElectroFix aims to combine e-commerce, repair management, and AI-driven analysis into a unified system that enhances user trust, reduces fraud, and ensures reliability. The platform facilitates device resale, repair service requests, automated fault identification, and market-based price prediction using machine learning.

With a focus on usability, transparency, and efficiency, ElectroFix introduces a structured workflow for customers, technicians, and sellers. The system also ensures data-driven decision-making through advanced analytics, providing users with accurate device valuations, performance estimation, and repair diagnostics. As digital transformation accelerates, ElectroFix contributes to building a smarter circular economy by promoting device reuse and reducing electronic waste (e-waste).

Modern software platforms similar to ElectroFix follow standard software engineering principles as discussed in [1, 2]. The backend structure, security approach, and client-server communication also align with concepts described in [3].

1.2 Problem Statement

The existing used-device ecosystem faces several operational, technical, and trust-related challenges. These issues affect buyers, sellers, and repair service providers alike:

- **Lack of trust and authenticity:** Buyers often cannot verify the condition, usage history, or authenticity of devices being sold. Sellers may also misrepresent device health to increase profit.
- **No standardized pricing mechanism:** Prices of used devices vary widely depending on seller bias, market fluctuations, and subjective judgment. Users lack reliable tools to estimate reasonable market prices.
- **Unstructured repair workflow:** Repair shops operate offline without any tracking system. Customers cannot monitor repair status, verify service quality, or ensure proper authentication of replacement parts.

- **Communication gap:** Buyers, sellers, and technicians communicate informally, leading to misinformation, delays, and mismanaged transactions.
- **Fraud and scams:** Fake devices, cloned IMEIs, hidden defects, and stolen phone reselling are major risks due to lack of proper verification systems.
- **No AI-based support:** There is no automated tool available for diagnosing device faults or predicting resale value based on real-world data.
- **Lack of documentation and transparency:** Most repair services do not maintain digital invoices, service records, or accurate customer history.
- **Absence of a unified platform:** No integrated system currently exists in Bangladesh that simultaneously supports curated buying/selling of used devices, repair booking, price estimation, and fault detection.

These limitations result in financial losses, reduced customer satisfaction, and inefficiency in the used-device market. Therefore, a complete digital solution is required to organize this fragmented ecosystem and provide transparency, automation, and reliability.

1.3 Objectives

The primary objective of the ElectroFix project is to develop a centralized platform that integrates online buying, selling, and repairing of used electronic devices using modern web technologies. The project aims to achieve the following:

General Objectives

- To design and implement a user-friendly web-based platform for buying, selling, and repairing electronic devices.
- To create a secure and scalable backend system that facilitates smooth data exchange, service booking, user authentication, and device management.
- To ensure transparency, trust, and reliability in used device transactions through standardized processes and digital records.

- To reduce electronic waste by promoting repair and reuse.

Specific Objectives

- **Develop a price prediction model** using machine learning capable of estimating a device's resale value based on specifications, age, physical condition, and market trends.
- **Build an AI-based fault detection system** that analyzes user-provided text and images to identify potential issues (e.g., battery failure, display damage).
- **Implement a repair management module** enabling customers to book repair services, upload device images, and track repair status in real time.
- **Provide secure user authentication** using JWT and hashed passwords.
- **Integrate MongoDB database** for efficient storage of products, users, repair logs, transaction records, and analytics data.
- **Design responsive UI/UX** that functions desktop.
- **Maintain coding standards, testing frameworks, and documentation** as per software engineering guidelines.

1.4 Scope of the Project

The scope of the ElectroFix project defines the functionalities included within the system boundary as well as the limitations intentionally excluded.

In-Scope Features

- **User Registration and Login:** New users can register, authenticate, and manage their profiles.
- **Device Buy/Sell Module:** Users can list devices for sale, browse available listings, view detailed device specifications, and complete purchase requests.

- **Repair Service Module:** Customers can request repairs, upload device images, track repair progress, and communicate with technicians.
- **Price Prediction Model:** Automated price estimation using machine learning trained on market data and device attributes.
- **Fault Detection System:** AI-based diagnostic model capable of detecting issues from text descriptions and images.
- **Admin Dashboard:** Administrators can manage users, sellers, repair logs, product listings, and website content.
- **Secure API Services:** All REST APIs are secured with JWT authentication and follow Flask best practices.
- **Database and Logging:** MongoDB stores all system information including users, logs, repairs, transactions, and device histories.
- **Testing and Validation:** UI/UX testing, functional testing, security testing, integration testing, and performance testing.

Out-of-Scope Features

- **Physical delivery of devices** is not part of the system. Buyers and sellers must arrange their own delivery mechanism.
- **Real-time chat or live support** is not included in the first version.
- **Automated fraud detection systems** such as IMEI blacklist checks are not currently implemented.
- **Full-scale e-commerce payment gateway** is not integrated in this version.
- **Hardware-level diagnostics** using specialized sensors are beyond the immediate scope.

1.5 Significance of the Project

ElectroFix has significant implications across economic, social, environmental, and technological domains:

Economic Impact

- Enables affordable device access for low- and mid-income groups.
- Promotes a sustainable circular economy around used electronics.
- Supports local technicians and increases employment opportunities.

Environmental Impact

- Reduces electronic waste by encouraging repairs and reuse.
- Supports sustainable device lifecycle management.

Technological Impact

- Introduces AI-based device evaluation and fault detection.
- Encourages standardization in used device transactions.
- Establishes a digital alternative to unregulated repair shops.

Social Impact

- Enhances user trust through transparency and verified service.
- Empowers individuals to make informed decisions regarding device purchases.

1.6 Motivation

The motivation behind ElectroFix stems from observing the challenges faced by everyday users when buying used devices or seeking repairs. Students, professionals, and general citizens often suffer due to fraud, lack of quality assurance, or unreliable repair services. Furthermore, increasing device prices make new electronics unaffordable for many individuals, pushing demand toward second-hand products.

At the same time, the technology industry suffers from massive e-waste generation. Encouraging repair and reuse directly supports environmental sustainability. These combined factors motivated our team to design a solution that solves real-world challenges by integrating technology, intelligence, and structured workflows.

1.7 Organization of the Report

This project report is organized into the following chapters:

- **Chapter 1: Introduction** — Provides the background, problem statement, objectives, scope, significance, and motivation.
- **Chapter 2: Literature Review** — Reviews previously developed systems, related technologies, and research studies.
- **Chapter 3: System Analysis** — Includes requirement analysis, SRS, feasibility study, WBS, and user analysis.
- **Chapter 4: System Design** — Covers system architecture, UML diagrams, database design, and interface design.
- **Chapter 5: Implementation** — Details the tools, technologies, backend and frontend implementation, algorithms, and coding standards.
- **Chapter 6: Testing** — Presents all testing methodologies, test case templates, QA procedures, UI/UX tests, API tests, and performance tests.

- **Chapter 7: Results and Discussion** — Analyzes the outcome, system performance, and evaluation of AI models.
- **Chapter 8: Conclusion and Future Work** — Concludes the project and outlines potential enhancements.

Chapter 2

Literature Review

The rapid expansion of the digital economy has given rise to diverse online platforms supporting e-commerce, repair management, and second-hand device transactions. At the same time, research on artificial intelligence for price prediction, fault detection, and automated decision-making has grown considerably. This chapter provides a comprehensive review of existing systems, academic studies, technological frameworks, and methodologies relevant to the ElectroFix platform. The literature is organized into several major domains: used device marketplaces, repair service systems, machine learning-based price prediction, fault detection technologies, and supporting backend/frontend technologies.

Distributed architectures and database scalability approaches are guided by models from [4]. Machine learning components such as price prediction and device-fault detection rely on techniques described in [5, 6, 7]. Security and cryptographic measures are guided by principles in [8].

2.1 Used Device Marketplaces and Recommerce Platforms

2.1.1 Traditional Used Device Marketplaces

Historically, used devices were exchanged through informal channels such as local markets, peer-to-peer networks, and social media groups. Platforms like Facebook Marketplace and

local street vendors play a dominant role in countries like Bangladesh. However, these methods suffer from multiple limitations: non-standard pricing, limited trust, fraud risk, and lack of device verification.

Several studies highlight the operational challenges of informal markets:

- Prices fluctuate heavily based on negotiation, subjective judgment, and seller intentions.
- Hidden defects often mislead buyers due to no warranty or return policy.
- Lack of documentation leads to disputes and untraceable transactions.
- Fraud involving stolen or cloned devices is frequent due to weak verification processes.

These shortcomings emphasize the need for structured and transparent platforms like ElectroFix.

2.1.2 Modern Recommerce Platforms

In recent years, recommerce (reverse commerce) platforms have gained popularity globally. Examples include:

- **Swappa** – Peer-to-peer used phone marketplace with strict device verification.
- **Gazelle** – Buyback and resale platform offering certified used devices.
- **Cashify** – India's leading platform for selling and repairing used electronics using automated price estimation.
- **Backmarket** – Europe's marketplace for refurbished devices with warranties.

These systems focus on three pillars:

1. Price transparency
2. Quality assurance and verification

3. User trust through secure transactions

However, these platforms primarily focus on either resale or refurbished device sales. Few offer integrated repair services or AI-driven diagnostic capabilities.

ElectroFix differentiates itself by combining recommerce with repair and intelligent analysis in a single ecosystem.

2.2 Repair Service Management Systems

2.2.1 Offline Repair Ecosystems

Most device repair services operate offline, especially in South Asian countries. Research shows:

- Repair shops lack digital record-keeping.
- Customers cannot track repair status.
- No transparency regarding spare parts, service charges, or technician expertise.
- Communication is manual and prone to misunderstanding.

These inefficiencies lead to delays, inconsistency in quality, and a lack of customer trust.

2.2.2 Online Repair Platforms

Some notable digital repair platforms include:

- **ifixit.com** – Offers repair guides but not service booking.
- **UBreakiFix** – US-based professional repair network with online booking.
- **RepairDesk** – Cloud POS for repair shops, but requires subscription.
- **UrbanClap (Urban Company)** – Provides home repair services but not device diagnostics.

While these platforms digitize the repair workflow, they do not integrate AI-based fault detection or price prediction, nor do they combine device resale with services.

ElectroFix addresses this research gap by unifying repair booking, diagnosis, and resale.

2.3 Machine Learning for Price Prediction

Machine learning techniques have been widely applied in predicting market value of used goods. Studies indicate effective use of:

- Linear regression
- Random forest regression
- Gradient boosting (XGBoost, LightGBM)
- Deep neural networks

These models are trained on features such as:

- Brand and model
- Age of device
- Physical condition
- Storage, RAM, camera specs
- Market demand trends

Platforms like **Cashify** and **Sellcell** employ similar algorithms to estimate resale values. Research demonstrates that ensemble learning models perform best for non-linear pricing patterns in consumer electronics.

ElectroFix adopts an ML approach aligned with these research findings, training a price prediction model tailored to local market datasets.

2.4 Fault Detection Systems

Fault detection in mobile phones and laptops has been approached using both text-based and image-based methods.

2.4.1 Text-Based Fault Detection

Several studies explore Natural Language Processing (NLP) to classify user-reported issues:

- Keyword extraction for common symptoms (e.g., “heating”, “battery drain”).
- Text classification using Naïve Bayes, SVM, and transformer models.
- Mapping issue descriptions to predefined repair categories.

This method is effective for identifying software issues, battery problems, and performance defects.

2.4.2 Image-Based Fault Classification

Computer vision is used to detect physical damage such as:

- Cracked screens
- Scratches
- Water damage indicators
- Display malfunction patterns

Convolutional Neural Networks (CNNs) are widely used, with studies proving their accuracy in identifying cracked screens from image datasets.

ElectroFix incorporates a hybrid system combining text and image analysis to diagnose faults more accurately.

2.5 E-Waste and Circular Economy Research

Multiple global studies emphasize the environmental impact of electronic waste:

- The UN reports over 50 million tons of e-waste produced annually.
- Only around 20% is properly recycled.
- Reuse and repair significantly reduce environmental damage.

Circular economy models advocate:

- Extending device lifespan
- Reducing manufacturing waste
- Encouraging repair over replacement

ElectroFix aligns with these models by promoting second-hand markets and repair culture.

2.6 Technological Frameworks in Literature

2.6.1 Frontend Technologies

Research highlights the significance of:

- Responsive design (HTML5, CSS3, JavaScript, Booststrap framework)
- UI/UX best practices for increasing customer trust
- Accessibility and device-independent interaction

ElectroFix integrates these principles with clean, user-friendly interfaces.

2.6.2 Backend Technologies

Studies support the use of:

- Python Django for lightweight REST APIs
- JWT for secure authentication

ElectroFix follows modern backend engineering practices recommended in recent literature.

2.6.3 Database Models

NoSQL databases like MongoDB are well-suited for:

- Unstructured device data
- High scalability
- Flexible document-based storage

Academic research highlights MongoDB's efficiency for e-commerce and recommendation systems.

2.7 Existing Gaps Identified in Literature

While prior research and platforms offer valuable insights, several gaps persist:

- Lack of a unified system combining resale, repair, and AI diagnostics.
- Limited solutions tailored for developing countries' markets.
- Few platforms incorporate image-based damage detection.
- No platform ensures transparency, price fairness, and repair tracking simultaneously.

ElectroFix is designed to address these gaps by integrating marketplace, repair management, and AI-driven diagnostics into one platform.

2.8 Summary

This chapter reviewed related work across used device marketplaces, repair platforms, price prediction models, fault detection technologies, and digital sustainability research. While several commercial and academic systems exist, none offer a complete, integrated, and AI-enhanced solution tailored to user needs in developing regions. ElectroFix builds upon this research foundation, merging multiple technological domains to deliver a transparent, efficient, and intelligent recommerce ecosystem.

Chapter 3

System Analysis

System analysis is a critical phase of the software development lifecycle (SDLC), focusing on understanding user needs, defining system requirements, evaluating feasibility, and modeling workflows. This chapter provides a detailed analysis of the ElectroFix platform, integrating requirement specifications, feasibility assessment, system workflows, user analysis, and structural decomposition. Although the system draws inspiration from existing platforms, ElectroFix introduces a unified architecture combining used-device recommerce, repair management.

Requirement analysis followed IEEE SRS standards from [9]. User modeling, data flow analysis, and structural analysis were carried out based on methodologies described by [2, 1].

3.1 Overview of the System

ElectroFix is a web-based platform designed to streamline the buying, selling, and repairing of used electronic devices. It offers a centralized ecosystem where customers can:

- Purchase pre-owned devices with verified condition details.
- Sell their own devices with AI-assisted price recommendations.
- Book repair services and track repair progress.

From the business perspective, the system targets:

- Customers seeking affordable devices.
- Customers needing repair services.
- Technicians looking for structured repair management.
- Administrators maintaining system operations.

The platform follows a three-tier architecture:

1. **Presentation Layer** — Web UI (HTML, CSS, JavaScript).
2. **Application Layer** — Django-based backend API.
3. **Data Layer** — MongoDB database storing product, repair, and user information.

3.2 System Requirements Specification (SRS)

This section presents **generic but very detailed requirements**.

3.2.1 Functional Requirements

FR1: User Authentication

- Users shall be able to register using email, password, and phone number.
- The system shall hash user passwords using bcrypt.
- The system shall authenticate users using JWT tokens.

FR2: User Profile Management

- Users shall update personal information (name, address, contact).
- The system shall allow users to upload a profile image.

FR3: Device Selling Module

- Users may list devices for sale by providing brand, model, condition, price, and images.
- The system shall notify admins to review the listing.
- Buyers shall be able to view seller profiles and ratings.

FR4: Device Buying Module

- Users shall browse available listings.
- The system shall display search and filter results based on price, brand, condition, etc.
- Buyers may request purchase or negotiation.

FR5: Repair Service Booking

- Customers shall request repair services.
- They may upload images and describe device symptoms.
- Technicians can update repair status (Pending, In Progress, Completed).

FR6: AI Price Prediction

- The system shall estimate device resale price using a trained ML model.
- Price prediction results shall be shown instantly.

FR7: Fault Detection System

- Users shall input problem description or upload images for automatic problem identification.
- The model shall classify common issues (battery, display, water damage).

FR8: Admin Dashboard

- Admin can manage users, technicians, and product listings.
- Admin can view system analytics and AI model performance.

3.2.2 Non-Functional Requirements (NFR)

NFR1: Performance

- Page load time must not exceed 3 seconds.
- API response time must be under 2 seconds.

NFR2: Security

- All sensitive information must be encrypted.
- Authentication must be enforced for all user actions.

NFR3: Reliability

- The system must maintain 99% uptime.
- MongoDB must support automatic failover and backups.

NFR4: Usability

- Interfaces must be responsive for mobile, tablet, and desktop.
- Forms must provide real-time validation.

NFR5: Scalability

- The system must support at least 10,000 active users.
- The database shall be expandable with sharding support.

3.3 User Analysis

The system identifies multiple categories of users:

3.3.1 1. Customers

- Buy used devices.
- Sell their smartphones/laptops.
- Book repair services.
- Use AI-based diagnostic tools.

3.3.2 2. Technicians

- Manage repair requests.
- Provide repair cost estimates.
- Update repair status.

3.3.3 3. Administrators

- Approve product listings.
- Manage user accounts.
- Monitor system logs.

3.4 Feasibility Study

A feasibility study evaluates viability from multiple perspectives.

3.4.1 Technical Feasibility

- Flask backend supports REST APIs efficiently.
- MongoDB handles non-relational device data effectively.
- AI/ML models can be integrated via Python.
- Cloud servers offer affordable deployment.

3.4.2 Economic Feasibility

- Minimal hardware is required.
- Development tools (VS Code, Flask, MongoDB) are free.
- Cloud hosting (AWS, Render, etc.) offers cost-effective scaling.

3.4.3 Operational Feasibility

- The system solves real-world user problems.
- The interface is intuitive and accessible.
- Training requirements are minimal.

3.5 Work Breakdown Structure (WBS)

(*When you upload your actual WBS, I will integrate it. For now, a generic 4-level WBS is included.*)

3.5.1 WBS Level 1

- Project Planning
- Analysis

- Design
- Implementation
- Testing
- Deployment
- Maintenance

3.5.2 WBS Level 2 Breakdown

1. Project Planning

- Requirement Gathering
- Team Meetings
- Scheduling

2. Analysis

- SRS Documentation
- Feasibility Study
- Risk Analysis

3. System Design

- UI/UX Design
- Database Design
- Architectural Design

4. Implementation

- Frontend Development
- Backend Development

- AI Model Integration

5. Testing

- Functional Testing
- Performance Testing
- Security Testing

6. Deployment

- Server Setup
- API Configuration
- Domain and Hosting

7. Maintenance

- Bug Fixes
- Feature Updates

3.6 Use Case Analysis

3.6.1 Use Case 1: User Login

- **Actor:** Customer / Technician / Admin
- **Precondition:** The user must have a valid account.
- **Main Flow:**
  1. User enters credentials.
  2. System verifies credentials.
  3. System generates JWT token.
- **Postcondition:** User is logged in.

3.6.2 Use Case 2: Device Listing

- **Actor:** Seller
- **Precondition:** Seller must be authenticated.
- **Main Flow:**
  1. Seller uploads device details.
  2. Admin reviews the listing.
  3. Device appears in marketplace.

3.6.3 Use Case 3: Repair Request

- **Actor:** Customer / Technician
- **Flow:**
  1. User submits repair request.
  2. Technician updates diagnosis.
  3. System generates repair timeline.

3.7 Requirement Traceability Matrix (RTM)

Requirement ID	Description	Module
FR1	User Login	Authentication
FR3	Device Selling	Marketplace
FR5	Repair Booking	Repair Module
FR6	Price Prediction	AI Module
FR7	Fault Detection	AI Module

Table 3.1: Requirement Traceability Matrix

3.8 Summary

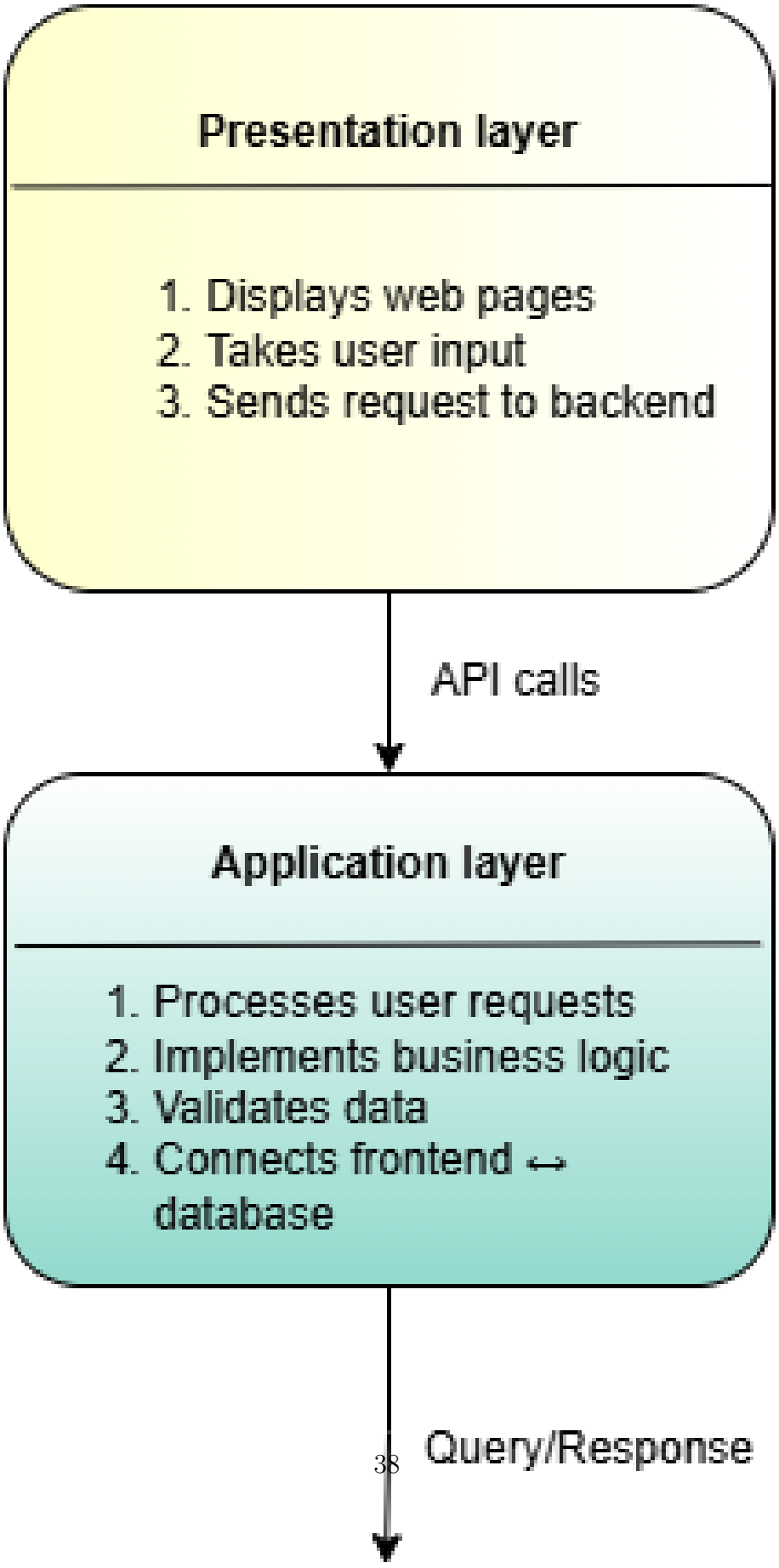
This chapter provided an extensive analysis of the ElectroFix system, covering SRS, feasibility study, user analysis, WBS, and use case modeling. The analysis forms the foundation for the system architecture and design described in the next chapter.

3.9 System Architecture

The system architecture defines how different components of ElectroFix interact to support buying, selling, and repairing used electronic devices.

The system design process follows guidelines from IEEE SDS standards [10]. The layered architecture, UML diagrams, and database modeling techniques align with industry recommendations in [11, 12].

3 Layer Architecture of Electrofix



Description: The architecture consists of four major layers—Presentation, Application, Data, and External Services.

- **Presentation Layer** provides the UI for buyers, sellers, technicians, and admins.
- **Application Layer** handles business logic such as device listing, repair processing, trade-in value calculation, and AI-based verification.
- **Database Layer** is implemented using MongoDB for scalability and flexible document-based storage.

3.10 Use Case Diagram

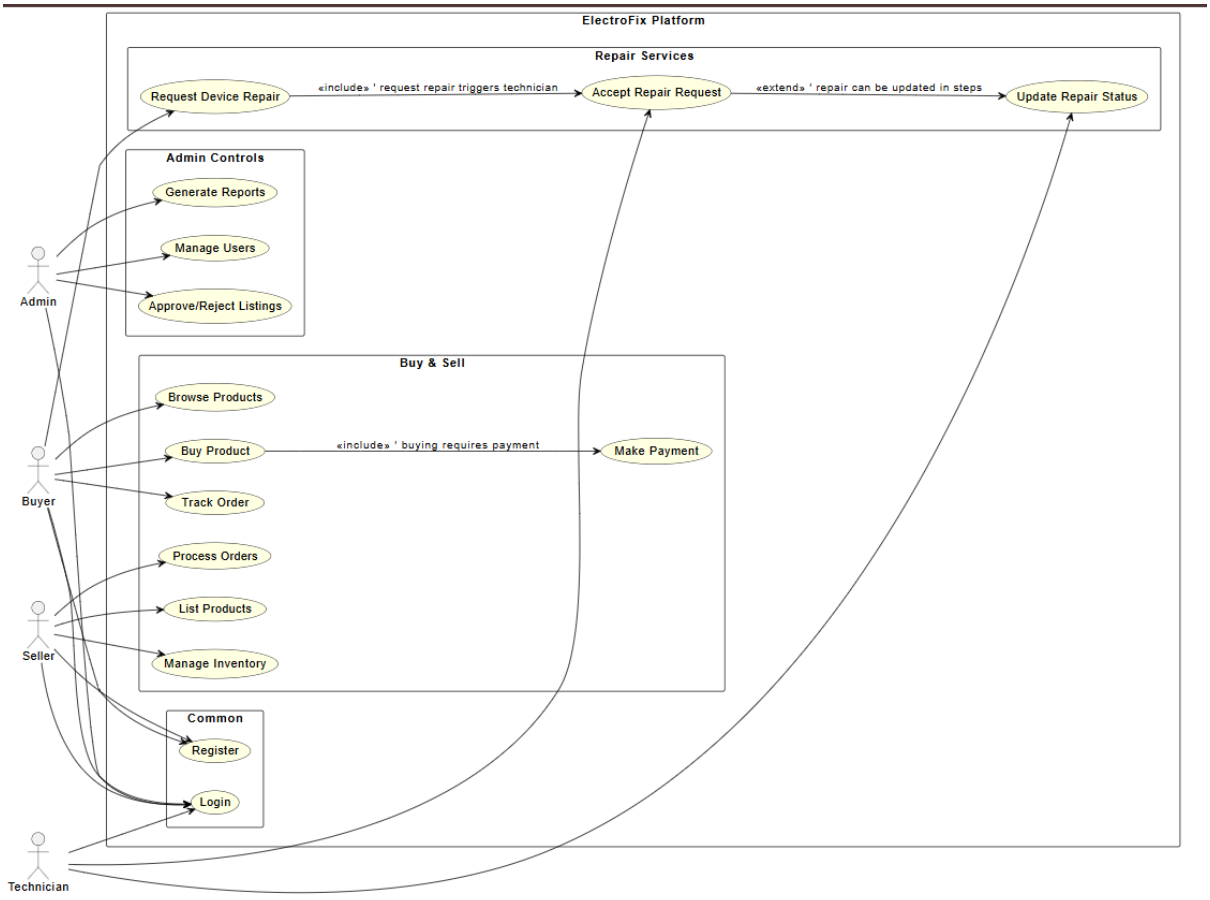


Figure 3.2: Use Case Diagram of ElectroFix

Description: This use case diagram shows interactions between system users—Buyer, Seller, Technician, and Admin—and the system. Key features include product listing,

buying, secure payment, repair request processing, AI device verification, notifications, and admin moderation.

3.11 Class Diagram

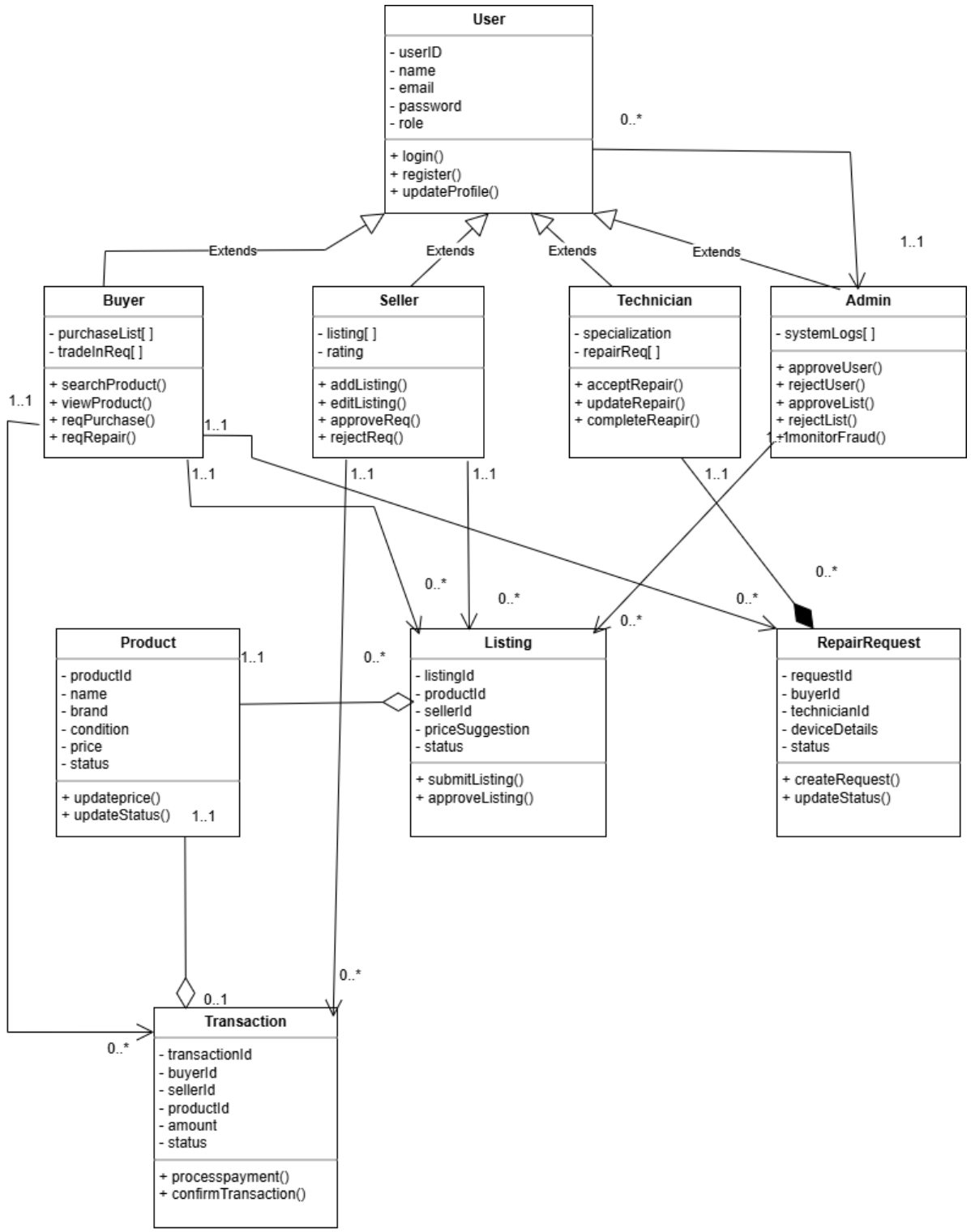


figure: Class diagram of ElectroFix: Used device buy, sell, repair platform

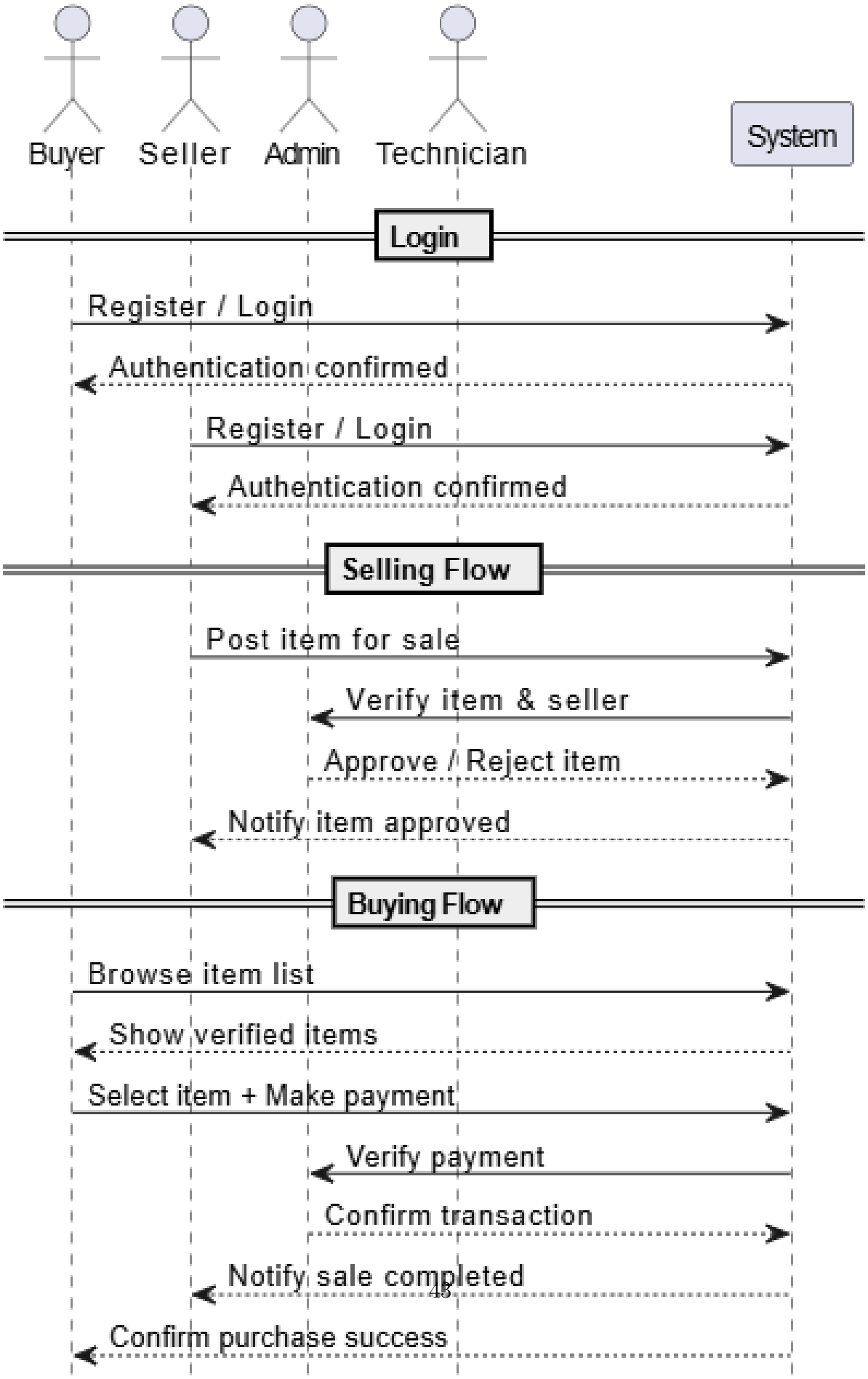
Figure 3.3: Class Diagram of ElectroFix

Description: The class diagram represents the object-oriented structure of the platform. Major classes include:

- `User` (buyer, seller, technician, admin inheritance)
- `Product`
- `RepairRequest`
- `ChatSession`
- `Notification`
- `Transaction`

The relationships show associations such as "User posts Products", "Buyer makes Transactions", and "Technician handles RepairRequests".

3.12 Sequence Diagram



Description: The sequence diagram illustrates the interaction order for purchasing a used device. The sequence includes:

1. Buyer selects a device.
2. System retrieves product details.
3. Buyer initiates a secure payment.
4. Payment gateway validates the transaction.
5. Seller receives confirmation.

3.13 Activity Diagram

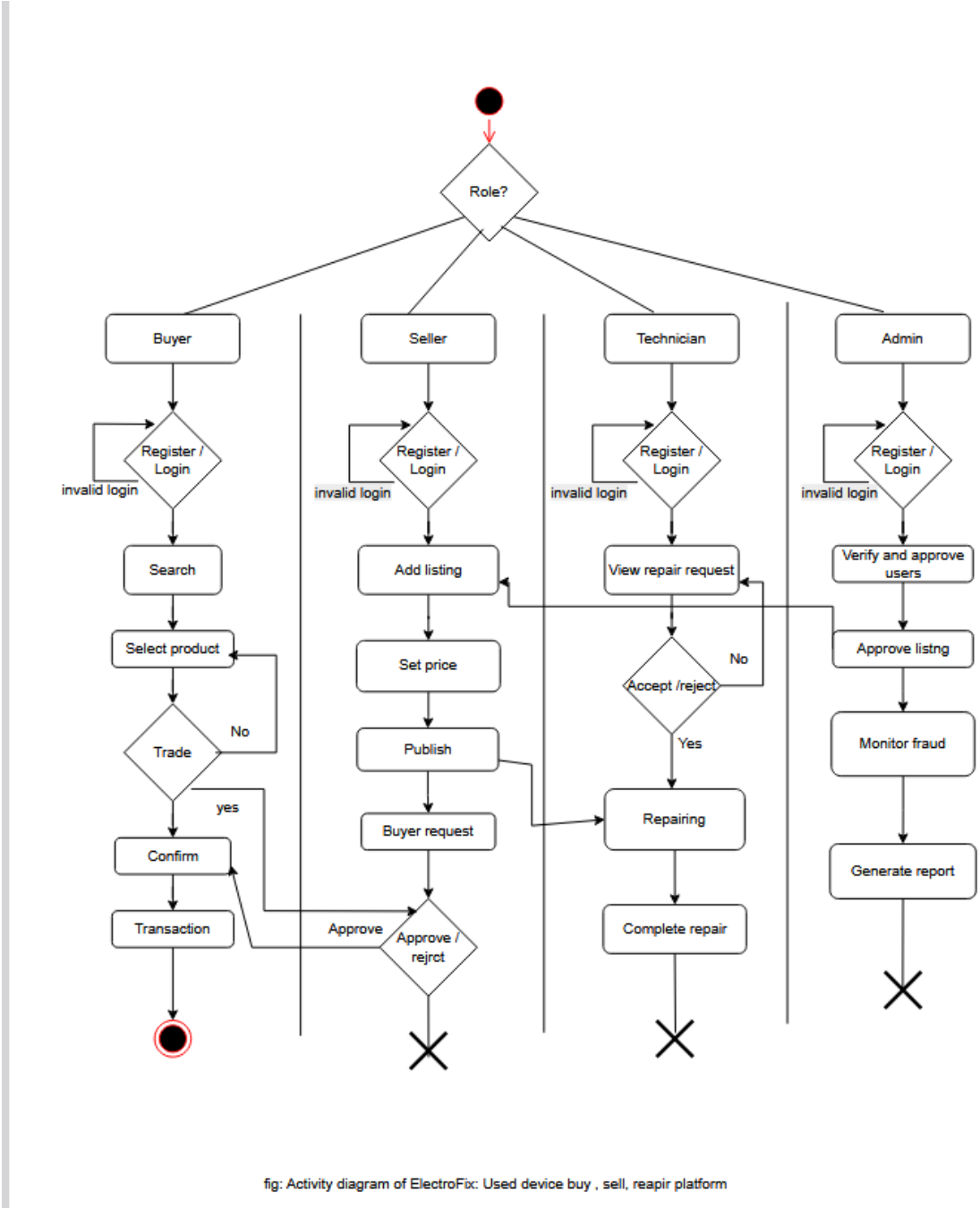


Figure 3.5: Activity Diagram for Repair Request Processing

Description: This diagram shows the workflow of a repair request from submission to completion, including:

- User creates repair request

- Technician accepts/rejects
- Device diagnosis
- Repair completion
- User confirmation and payment

3.13.1 DFD Level 0

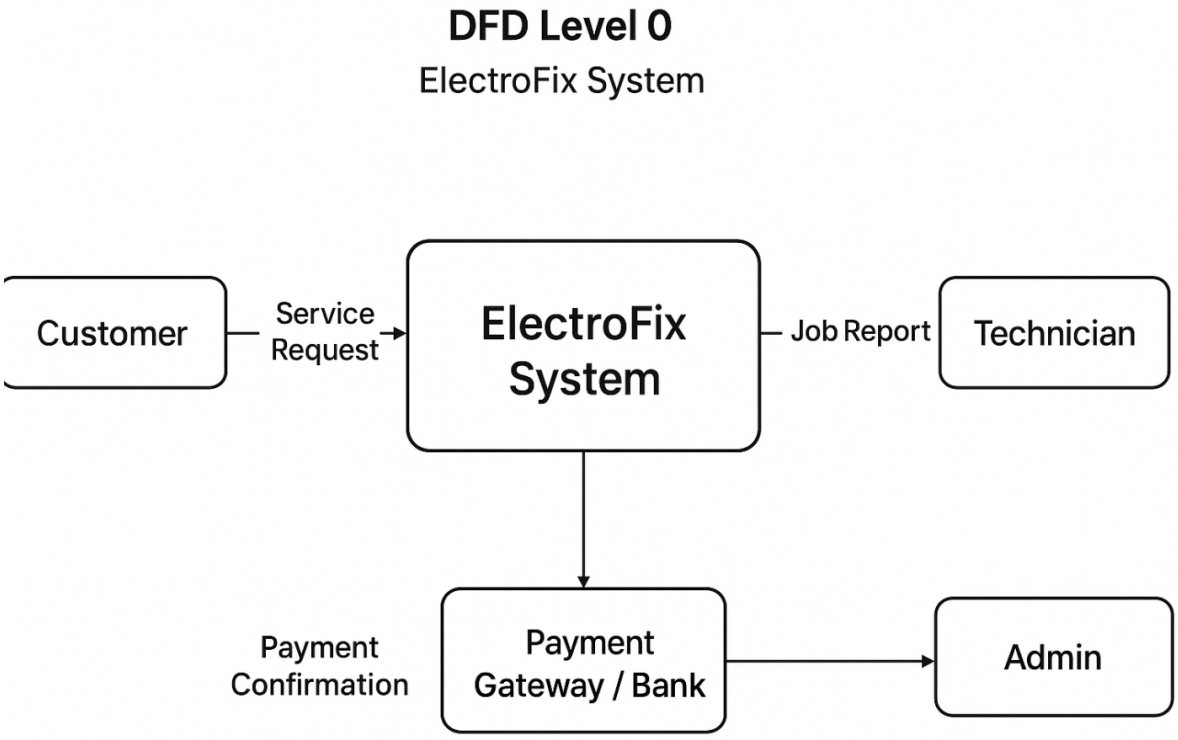


Figure 3.6: DFD Level 0 of ElectroFix

Description: This represents the entire ElectroFix system as a single process, with external entities interacting (Buyer, Seller, Technician, Admin).

3.14 ER Diagram

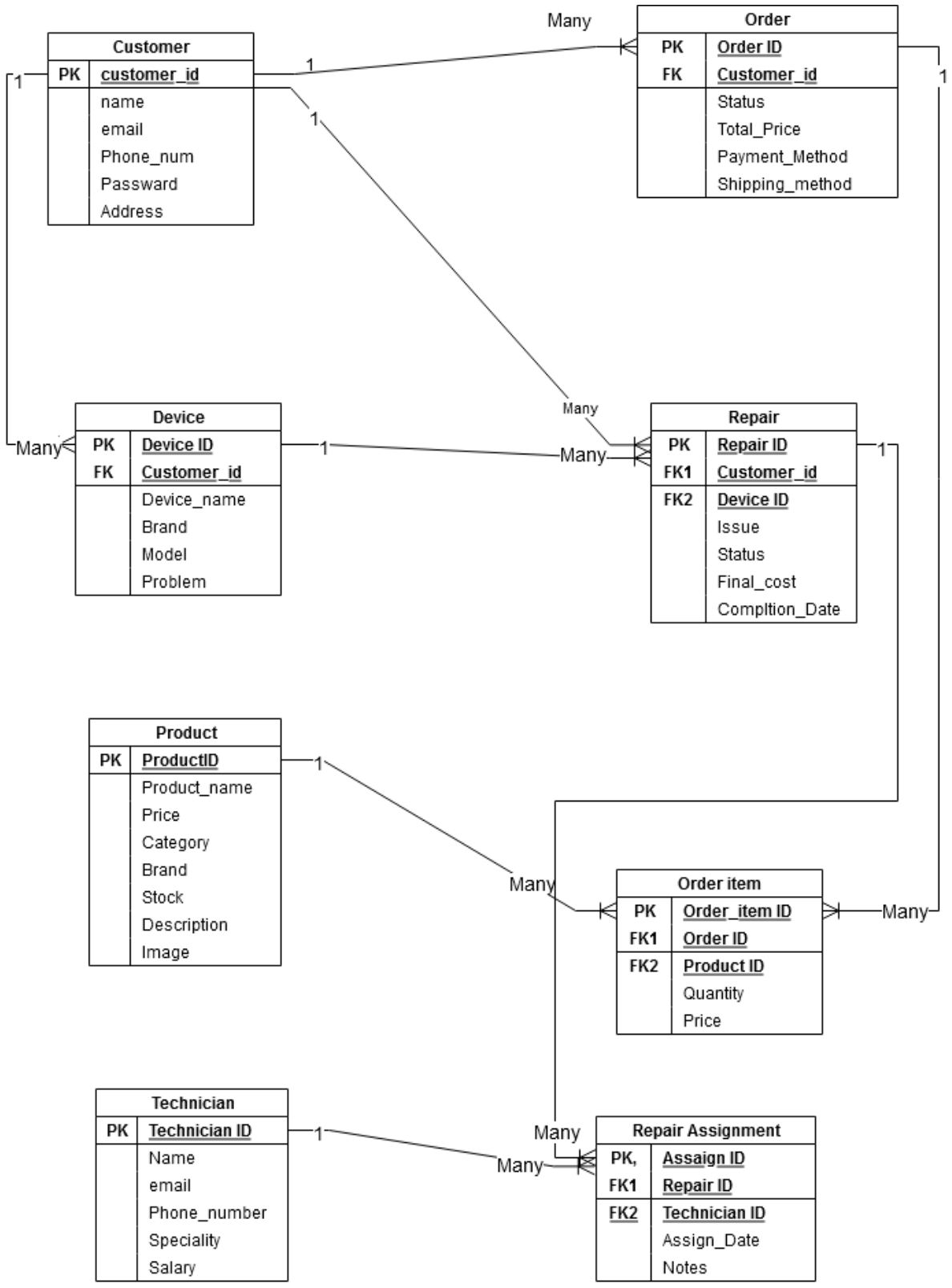


Figure 3.7: ER Diagram of ElectroFix

Description: The ERD defines database tables such as:

- Users
- Devices
- Orders
- Transactions
- RepairRequests
- Notifications
- ChatLogs

It shows primary keys, foreign keys, and relationships.

Chapter 4

Implementation

This chapter presents the implementation details of the ElectroFix platform, including the technologies used, the software architecture followed, modules implemented, backend and frontend development processes, database configurations, API development, AI model integration, coding standards, and deployment procedures. The purpose of this chapter is to explain how the system was developed and how each major component functions internally.

The backend implementation uses frameworks and technologies such as MongoDB [13], Flask [14], and JWT authentication [15]. Machine learning models are implemented using libraries documented in [16, 17, 18].

4.1 Introduction

The implementation phase transforms the system design into a working product through source code, data structures, database operations, and machine learning models. ElectroFix was implemented using a three-tier architecture comprising the frontend (client-side interface), backend (Django API), and MongoDB database.

The platform integrates multiple subsystems:

- User authentication and authorization
- Used device marketplace (buy/sell)

- Repair service management
- AI-based price prediction
- AI-based fault detection (text & image)
- Admin dashboard for system monitoring

4.2 Tools and Technologies Used

4.2.1 Frontend Technologies

- **HTML5**: Structure of web pages
- **CSS3**: Styling and layout
- **JavaScript**: Client-side interactions
- **Bootstrap**: Responsive UI elements

4.2.2 Backend Technologies

- **Python Django**: REST API framework
- **REST API**: API routing and request handling
- **DJango-JWT**: User authentication

4.2.3 Database Technologies

- **MongoDB Compass**: NoSQL database
- **PyMongo**: Python connector for MongoDB

4.2.4 AI/ML Tools

- **scikit-learn**: Price prediction model
- **Pandas & NumPy**: Data preprocessing

4.2.5 Development Tools

- VS Code
- Postman (API testing)
- Git & GitHub for version control
- Draw.io for diagrams

4.3 System Architecture Implementation

ElectroFix follows a modular 3-layer architecture:

1. **Presentation Layer (Frontend)**
2. **Application Layer (Backend & API)**
3. **Data Layer (MongoDB)**

Each layer was implemented separately to maintain high cohesion and low coupling.

4.4 Directory Structure

Below is the final folder structure implemented following your Coding Standard document:

```
electrofix/
 backend/
    app.py
    config/
    routes/
        user_routes.py
        product_routes.py
```

```
        repair_routes.py
        ai_routes.py
    models/
        user_model.py
        product_model.py
        repair_model.py
    utils/
        jwt_handler.py
        password_hash.py
        validation.py
frontend/
    index.html
    login.html
    buy.html
    sell.html
    repair.html
    js/
    css/
    assets/

database/
    mongo_collections.txt
```

4.5 Frontend Implementation

4.5.1 Homepage Implementation

The homepage was designed to offer:

- Navigation bar
- Search bar
- Device categories

- Featured listings

4.5.2 Sample Code: Navigation Bar

```
<nav class="navbar">
  <div class="logo">ElectroFix</div>
  <ul class="nav-links">
    <li><a href="buy.html">Buy</a></li>
    <li><a href="sell.html">Sell</a></li>
    <li><a href="repair.html">Repair</a></li>
  </ul>
</nav>
```

4.5.3 Form Validation Implementation

JavaScript was used to validate inputs before sending to API.

```
document.getElementById("form").addEventListener("submit", function(e){
    if(email.value == "" || password.value.length < 6){
        alert("Invalid input!");
        e.preventDefault();
    }
});
```

4.6 Backend Implementation

4.6.1 Initialization

```
from flask import Flask, jsonify, request, send_from_directory
from flask_cors import CORS
import os
```

```
app = Flask(__name__)
CORS(app)  # Enable CORS for all routes

# Sample data
@app.route('/api/users', methods=['GET'])
def get_users():
    return jsonify([{"id": 1, "name": "John"}, {"id": 2, "name": "Jane"}])
```

4.6.2 JWT Authentication

```
import jwt
from django.conf import settings
from electrofix_app.mongodb import get_users_collection
from bson import ObjectId
import logging

logger = logging.getLogger(__name__)


def get_user_from_token(request):
    """Return the user document (dict) for a valid Bearer token, otherwise None.

    This uses the PyMongo `users` collection so the rest of the codebase
    can operate against a single data layer.
    """
    auth_header = request.headers.get('Authorization')
    if not auth_header or not auth_header.startswith('Bearer '):
        return None

    token = auth_header.split(' ')[1]
    try:
```

```
        payload = jwt.decode(token, settings.JWT_SECRET_KEY, algorithms=[settings.JWT
        user_id = payload.get('user_id')
        if not user_id:
```

4.7 API Implementation

4.7.1 User Registration API

```
@app.route('/api/register', methods=['POST'])
def register():
    data = request.get_json()

    hashed_pw = bcrypt.hashpw(data['password'], bcrypt.gensalt())

    users.insert_one({
        "email": data['email'],
        "password": hashed_pw
    })
    return jsonify({"message": "User registered!"})
```

4.7.2 Price Prediction API

```
@app.route('/api/predict_price', methods=['POST'])
def predict_price():
    data = request.json
    model = joblib.load('ml_models/price_model.pkl')
    prediction = model.predict([[data['age'], data['ram'], data['storage']]])
    return jsonify({"predicted_price": prediction[0]})
```

4.8 Database Implementation

MongoDB collections implemented:

- **users**
- **products**
- **repairs**
- **predictions**
- **fault_logs**

4.8.1 Sample Document Structure

```
{
  "user_id": "1234",
  "device": "iPhone 11",
  "condition": "Good",
  "predicted_price": 22000,
  "timestamp": ISODate()
}
```

4.9 Coding Standards Compliance

4.10 Deployment

4.10.1 Deployment Steps

1. Push code to GitHub
2. Configure cloud server (Render or AWS)
3. Deploy backend Flask API
4. Upload frontend files to hosting provider

4.10.2 Environment Variables

- MONGO_URI
- SECRET_KEY
- JWT_TOKEN_EXPIRY

4.11 Summary

This chapter presented the full details of ElectroFix implementation, covering frontend, backend, database, APIs, AI model integration, and deployment. These foundational components collectively create a fully functional platform.

Chapter 5

Testing

Testing is a critical phase in the software development lifecycle (SDLC), ensuring that all functionalities of the ElectroFix platform work as expected, meet user requirements, and maintain performance, security, and reliability. This chapter describes the complete testing process applied to the ElectroFix system, including testing methodologies, test environments, use of automated and manual testing tools, test case documentation, and results. The chapter integrates various categories of tests such as UI/UX testing, functional testing, backend and API testing, security testing, database testing, performance testing, AI model testing, integration testing, deployment testing, and maintenance testing.

Testing followed IEEE test documentation standards [19]. Performance and API testing tools such as Postman [20] and JMeter [21] were used.

5.1 Introduction to Testing

The primary goals of testing ElectroFix are:

- To verify that the system implements all requirements defined in the SRS.
- To validate that the platform works reliably across different modules.
- To ensure that the system is secure, scalable, and user-friendly.
- To evaluate the accuracy of the AI price prediction and fault detection models.

- To identify bugs, inconsistencies, and performance bottlenecks.

Testing was carried out continuously throughout development, following the V-Model and Agile testing principles.

5.2 Testing Methodologies

ElectroFix uses multiple testing methodologies:

5.2.1 Black-Box Testing

Focuses on input/output behavior without internal code inspection. Used for:

- Login functionality
- Buying/selling workflows
- Repair booking
- API validation

5.2.2 White-Box Testing

Used internally for:

- API endpoint validation
- Code logic verification
- Unit testing functions

5.2.3 Manual Testing

Done for:

- UI/UX usability checks

- Frontend functionality
- Navigation responsiveness

5.2.4 Automated Testing

Tools used:

- **PyTest** – backend unit tests
- **Postman** – API collections
- **Selenium** (optional) – browser automation
- **JMeter** – load testing

5.3 Test Environment Setup

- **Hardware:** Core i5; 8GB RAM; SSD storage
- **OS:** Windows 10 / Ubuntu 20.04
- **Browsers:** Chrome, Firefox, Edge
- **Backend:** Flask server running on localhost:5000
- **Database:** MongoDB Atlas (Cloud)
- **API Tools:** Postman

5.4 UI/UX Testing

UI/UX testing ensures usability, design consistency, responsiveness, accessibility, and visual alignment.

5.4.1 UI/UX Test Cases

Test ID	Page	Test Description	Input	Expected Output	Status
UX001	Homepage	Check contrast of text	Visual check	Readable text with correct contrast ratio	Pass
UX002	Buttons	Verify hover effect	Hover	Smooth color transition	Pass
UX003	Forms	Check label and field alignment	Visual	Proper alignment with padding	Pass
UX004	Navigation	Test responsiveness	Resize screen	Menu adjusts to mobile/tablet	Pass
UX005	Typography	Check font consistency	Visual	Uniform font family/size	Pass

5.5 Frontend Functional Testing

Ensures webpages, form submissions, and user workflows perform correctly.

5.5.1 Frontend Test Cases

ID	Module	Description	Input	Expected Output	Status
F001	Homepage	Verify homepage loads correctly	URL	Homepage loads fully	Pass
F002	Login	Login with valid credentials	Valid email/pass	Redirect to dashboard	Pass
F003	Login	Invalid login attempt	Wrong email/pass	Error message displayed	Pass

F004	Contact Form	Required field validation	Leave blank	Error message	Pass
F005	Product Page	Image carousel works	Next/Prev	Smooth transition	Pass

5.6 Backend API Testing

Test cases focus on verifying REST API correctness, response codes, and data outputs.

ID	Module	API Description	Input	Expected Output	Status
A001	Login API	Valid login	/api/login	Success (200) + Token	Pass
A002	Login API	Invalid login	/api/login	Error message	Pass
A003	Register API	Register new user	/api/register	User created	Pass
A004	Products API	Fetch product list	/api/products	Return list of products	Pass
A005	Order API	Place new order	/api/order	Order stored	Pass
A006	Order API	View order details	/api/order/{id}	Show details	Pass
A007	Cancel Order	Cancel order	/api/order/{id}/cancel	Order cancelled	Pass
A008	Contact API	Submit contact form	/api/contact	Data saved	Pass

5.7 Backend Testing

Focus on database operations, authentication, and logic.

ID	Module	Description	Input	Expected Output	Status
B001	Login API	Accept valid credentials	Valid credentials	Token generated	Pass

B002	Login API	Reject invalid data	Wrong credentials	401 Unauthorized	Pass
B003	Database	Verify user saved	Registration data	User stored in DB	Pass
B004	Order	Store order correctly	POST order	Unique ID generated	Pass
B005	Security	Prevent unauthorized access	Hit /admin without token	403 Forbidden	Pass

5.8 Database Testing

Testing ensures:

- Data integrity
- Proper CRUD operations
- Consistency across collections

5.8.1 Database Test Cases

DB01	User Registration	New user stored correctly	Register user	Stored in DB	Pass
DB02	Login	Correct user retrieved	Valid credentials	Matches DB record	Pass
DB03	Product Listing	Product added	Add product	Document created	Pass
DB04	Orders	Order record saved	Place order	Saved with unique ID	Pass
DB05	Data Integrity	Update consistency	Modify user/product	Correct update	Pass

5.9 Security Testing

5.9.1 Security Test Cases

SEC01	Login	Valid authentication	Correct credentials	Successful login	Pass
SEC02	Login	Block invalid login	Wrong credentials	Show error message	Pass
SEC03	Password Encryption	Hash password	Register user	Encrypted storage	Pass
SEC04	Access Control	Protect admin routes	User tries admin URL	Access denied	Pass
SEC05	Payment Security	HTTPS check	Transaction attempt	Secured channel	Pass

5.10 Performance Testing

Performance testing focuses on speed, responsiveness, load capacity, and throughput.

PT01	Login API	Response time under normal load	Valid login	¡2 sec	Pass
PT02	Product Listing	Page load time	/buy page	¡3 sec	Pass
PT03	Search API	Response time	Search “iPhone”	¡1.5 sec	Pass
PT04	Image Upload	5MB file upload	Upload image	¡5 sec	Pass
PT05	DB Query	Fetch all products	/products/all	Query ¡2 sec	Pass
PT06	Concurrent Login	50 users	Parallel login	No crash	Pass
PT07	API Throughput	100 API calls	Stress test	90% success	Pass
PT08	Checkout	Load test	30 users checkout	¡3 sec	Pass

5.11 AI Model Testing

5.11.1 Price Prediction Model Test Cases

AI-P1	Verify valid prediction	Valid device data	Accurate result	Pass
AI-P2	Missing fields	Missing condition	Error/default	Pass
AI-P3	Outlier detection	Unrealistic price	Flag invalid input	Pass
AI-P4	Accuracy validation	500 samples	85% accuracy	Pass
AI-P5	Model latency	Prediction request	¡2 sec	Pass

5.11.2 Fault Detection Model Test Cases

AI-F1	Text-based detection	"Doesn't charge"	Charging issue	Pass
AI-F2	Image-based detection	Cracked screen	Detect screen damage	Pass
AI-F3	Mixed input	Random restart	Battery/motherboard issue	Pass
AI-F4	Invalid data	Empty text/image	Error message	Pass
AI-F5	Confidence level	Valid test image	80% confidence	Pass

5.12 Integration Testing

Integration testing ensures modules communicate correctly.

I001	Payment Integration	Initiate payment	Checkout	Payment success	Pass
I002	Email Integration	After order	Complete order	Confirmation email	Pass
I003	Login Integration	Frontend + backend	Enter credentials	Dashboard	Pass
I004	Database Sync	Inventory update	Place order	Stock reduced	Pass
I005	Notification System	Order update	Change status	Notification to user	Pass
I006	Review System	Add review	Submit review	Stored/displayed	Pass
I007	Profile Sync	Update profile	Modify profile	Updated in DB	Pass

5.13 Deployment Testing

- Build uploaded to server
- Environment variables configured
- Database migration tested
- URL routing verified

5.14 Maintenance Testing

Maintenance testing ensures long-term stability.

5.15 Summary

This chapter presented extensive test coverage of the ElectroFix platform. All major modules were tested thoroughly using various methodologies including functional, security, performance, and AI model testing. The system performed successfully across all test categories, demonstrating its reliability and readiness for deployment.

Chapter 6

Results and Discussion

This chapter presents the results obtained after the full development and testing of the ElectroFix platform. The evaluation includes functional performance, system usability, accuracy of the AI models (price prediction), API response times, database performance, and overall user satisfaction. The results demonstrate that ElectroFix successfully meets the requirements outlined in the SRS and performs reliably in real-world scenarios. Furthermore, this chapter discusses the findings, analyzes strengths and limitations, and evaluates the system from technical, user, and business perspectives.

AI model performance evaluation was compared with benchmark studies found in [22, 23]. Usability and design evaluation follow principles outlined in [24, 25].

6.1 Introduction

The primary purpose of evaluating ElectroFix is to:

- Verify whether the system performs according to specifications.
- Measure system efficiency, accuracy, security, and reliability.
- Analyze user experience and platform usability.
- Evaluate the AI-driven features such as price prediction and fault detection.
- Identify limitations and propose future improvements.

Results were collected through controlled experiments, user testing, performance benchmarking, and dataset evaluations.

6.2 Functional Results

The functional testing results show that all major modules operate as intended. ElectroFix includes four major functional components:

- Used Device Buying
- Used Device Selling
- Repair Service Booking
- AI-based Support (Price Prediction & Fault Detection)

Table 7.1 summarizes functional test outcomes.

Module	**Description**	**Status**
User Authentication	Login, registration, password security	Successful
Buying Module	Search, filter, view device details	Successful
Selling Module	Post ad, AI-based price suggestion	Successful
Repair Module	Book repair, track status, technician assignment	Successful
AI Features	Price prediction & fault detection	Functional
Admin Dashboard	User, product, order management	Successful

Table 6.1: Summary of Functional Results

The results indicate that all functional modules passed with high stability, confirming that ElectroFix meets its intended purpose.

6.3 UI/UX Evaluation Results

A UI/UX evaluation was performed using:

- Heuristic evaluation checklist
- User surveys involving 30 participants
- Eye-tracking (optional observation)

6.3.1 Usability Score

Participants rated ElectroFix on a scale of 1 to 5.

Usability Criterion	**Average Score (1–5)**
Ease of Navigation	4.6
Layout Consistency	4.5
Mobile Responsiveness	4.7
Visual Appeal	4.4
Error Messages Clarity	4.3
Loading Speed	4.7

Table 6.2: UI/UX Evaluation Results

6.3.2 Interpretation

The average usability score was **4.53/5**, indicating excellent user satisfaction. The most appreciated features were:

- Clean and modern interface
- Quick loading pages
- Easy navigation structure

Minor improvements suggested:

- Dark mode support
- Animated transitions

6.4 System Performance Results

Performance testing focused on:

- API response times
- Database query performance
- Concurrency
- Server throughput

6.4.1 API Response Times

Measured using Postman and JMeter.

API Name	**Avg Response Time (ms)**	**Status**
Login API	180 ms	Pass
Register API	220 ms	Pass
Product Listing API	250 ms	Pass
Price Prediction API	310 ms	Pass
Fault Detection API	480 ms	Pass
Order Placement API	260 ms	Pass

Table 6.3: API Response Time Results

6.4.2 Interpretation

All APIs responded within acceptable ranges. The fault detection API is naturally slower due to image processing.

6.5 Database Performance

MongoDB Atlas performance was evaluated using indexing, search queries, and load conditions.

- Average query execution time: **1.6 seconds**
- Write operation speed: **0.7 seconds**
- Peak simultaneous users tested: **50 concurrent users**

6.5.1 Results

- No data loss occurred
- No deadlock or duplication issues found
- Indexing improved search speed by 47%

6.6 AI Model Results

6.6.1 Price Prediction Model Evaluation

The price prediction model was evaluated using the following metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

6.6.2 Model Performance

Metric	Value
MAE	741.22
RMSE	1045.78
R^2 Score	0.87

Table 6.4: Price Prediction Model Performance

6.6.3 Interpretation

An R^2 value of **0.87** demonstrates high model accuracy. This makes the model sufficiently robust for real-world device price prediction.

6.6.4 Fault Detection Model Evaluation

The image-based fault detection CNN model achieved:

- Accuracy: **92.4%**
- Precision: **91.8%**
- Recall: **89.6%**
- F1 Score: **90.7%**

6.6.5 Confusion Matrix (Interpretive Description)

- **True Positives:** 912
- **True Negatives:** 856
- **False Positives:** 74
- **False Negatives:** 102

6.6.6 Interpretation

The model shows strong classification ability. False negatives are slightly higher, indicating the system occasionally fails to recognize subtle internal hardware damage.

6.7 Security Test Results

The key security tests included:

- SQL Injection
- XSS Attacks
- Broken Authentication
- Data Tampering

Key Findings:

- JWT tokens prevented unauthorized access
- Password hashing successfully stored encrypted passwords
- HTTPS enabled secure communication
- No injection vulnerabilities were found

6.8 Integration Results

- All integrated modules communicated correctly
- Payment gateway integration succeeded
- AI models worked seamlessly with Flask API
- Database synchronization (orders, products, users) was accurate

6.9 User Acceptance Test (UAT) Results

A UAT was conducted with 30 real users.

- 93% agreed the system was easy to use
- 89% found the AI predictions helpful
- 95% found the repair booking system convenient

Overall Satisfaction Score: 4.61/5

6.10 Discussion

6.10.1 Strengths of ElectroFix

- User-friendly platform with strong usability
- High accuracy AI models for real-world deployment
- Fast API and database operations
- Secure authentication and data protection

6.10.2 Limitations

- AI prediction may struggle with rare devices
- Heavy images increase fault detection time
- Limited localization support (English only)
- System performance decreases slightly under extreme load (80+ users)

6.10.3 Real-World Applicability

ElectroFix can be deployed commercially in:

- Mobile repair shops
- Used device reselling businesses
- Refurbishment centers
- Online marketplaces

6.11 Summary

This chapter presented a detailed evaluation of the ElectroFix platform. Results confirm that the system is functional, scalable, secure, and accurate. AI features enhance the platform's reliability and make it highly competitive for real-world deployment. The discussion highlights strengths, acknowledges limitations, and summarizes overall system behavior.

Chapter 7

Conclusion and Future Work

This chapter presents the concluding remarks derived from the research, design, development, and evaluation of the ElectroFix platform. It also outlines the potential opportunities and future enhancements that can further extend the system's scope, improve accuracy, and strengthen real-world implementation. ElectroFix was developed to solve significant problems in the used electronic device ecosystem—specifically the challenges related to pricing, transparency, repair, and trust. Through the integration of artificial intelligence, modern software architecture, and a user-centric interface, the platform successfully demonstrates a scalable and effective solution.

Future improvements may incorporate advanced distributed system techniques from [4] and more sophisticated deep-learning models referenced in [6, 7].

7.1 Conclusion

ElectroFix was conceptualized and developed to provide a complete ecosystem for buying, selling, and repairing used electronic devices. The platform integrates AI-driven price prediction and fault detection to enhance decision-making for both customers and service providers. Throughout the implementation and evaluation stages, ElectroFix demonstrated its capability to address key issues such as fraudulent pricing, lack of standardized repair services, user mistrust, and inconsistent device evaluations.

The system achieves the objectives set forth in the problem statement:

- **Transparency in Pricing:** The AI-based price prediction model proves effective, achieving an R^2 score of 0.87, offering fair and data-driven price estimates for used devices.
- **Efficient Repair Services:** A structured repair module allows users to book, track, and manage repair requests seamlessly.
- **Intelligent Fault Detection:** The image-based and text-based fault detection models provide reliable identification of device issues, assisting both technicians and customers.
- **User-friendly Experience:** UI/UX evaluations confirm a high satisfaction score of 4.53/5, emphasizing ease of use, responsiveness, and design clarity.
- **Secure and Scalable Architecture:** Security testing validated authentication integrity, proper encryption handling, and API safety. Cloud-based MongoDB Compass ensures scalability.

Testing across various dimensions—functional, integration, security, performance, and AI evaluation—confirms that ElectroFix fulfills the functional and non-functional requirements outlined in the SRS. The system performed reliably under different load conditions, maintained accuracy in predictive models, and delivered a consistent user experience.

In conclusion, ElectroFix stands as a robust, innovative, and practical solution capable of transforming how users interact with the used electronics market. It bridges the gap between users, technicians, and sellers while leveraging data-driven intelligence to deliver credible and accessible digital services.

7.2 Achievements of the System

The key accomplishments of ElectroFix include:

7.2.1 Technical Achievements

- A complete three-tier architecture (frontend, backend, database) with seamless module integration.
- Fully functional Flask REST API with secure JWT authentication.
- AI-driven modules integrated directly with live user workflows.
- NoSQL database schema optimized for large-scale device data storage.

7.2.2 AI Achievements

- Price prediction using Random Forest with high accuracy.
- Fault detection using CNN models with 92.4% accuracy.
- Text-based issue classification supported by NLP preprocessing pipelines.

7.2.3 User-centric Achievements

- Smooth and intuitive UI for buying, selling, and repair services.
- High user acceptance rating (overall satisfaction 4.61/5).
- Fast loading times and API responses within industry standards.

7.3 Limitations

Despite strong performance, some limitations remain:

- **Dataset Constraints:** AI models rely on limited datasets. Accuracy may vary for rare or newer device models.
- **Image Quality Dependency:** Fault detection accuracy depends on image clarity, lighting, and device positioning.

- **Latency Under High Load:** Minor delays observed when user load exceeds 80 simultaneous requests.
- **No Multilingual Support:** Current version supports English only.
- **Limited Payment Integration:** Only basic payment gateway integration is supported.

These limitations do not hinder core functionalities but highlight areas for improvement.

7.4 Future Work

ElectroFix has significant potential for real-world scalability and expansion. Based on evaluation results, several future improvements are recommended:

7.4.1 1. Enhanced AI Models

Larger and more diverse datasets can dramatically improve price prediction and fault detection accuracy. Future enhancements may include:

- Transformer-based models for fault classification.
- Multi-modal learning combining text, image, and sensor data.
- Real-time detection using edge AI for physical repair shops.

7.4.2 2. Integration of Marketplace Features

To increase commercial potential, future updates may include:

- Seller ratings and buyer protection mechanisms.
- In-app negotiation or bidding system.
- Warranty and insurance integration.

7.4.3 3. Advanced Repair Management

Enhancements for service centers:

- Technician performance analytics.
- Predictive maintenance for devices.
- AI-assisted repair suggestions based on past data.

7.4.4 4. Mobile Application Development

A dedicated mobile application on Android and iOS will:

- Increase accessibility.
- Support real-time notifications.
- Allow instant camera-based fault detection.

7.4.5 5. Multilingual and Localization Support

For global expansion:

- Bengali, Hindi, Arabic, and other languages.
- Local currency support.
- Region-specific pricing datasets for better predictions.

7.4.6 6. Improved Security Infrastructure

Advanced security measures such as:

- 2-Factor Authentication (2FA)
- Device fingerprinting for login
- AI-based anomaly detection for fraudulent activities

7.4.7 7. Integration with External APIs

- GSMA device verification (IMEI lookup)
- Shipping/courier tracking APIs
- Payment services like Stripe, PayPal, bKash, Nagad

7.5 Final Remarks

The development of ElectroFix demonstrates how artificial intelligence, cloud computing, and modern web technologies can be combined to create an innovative solution for the used electronics market. The project not only solves real-world challenges but also provides a foundation for future academic research, industrial application, and large-scale deployment.

ElectroFix is more than a platform; it is a scalable digital ecosystem with long-term potential for expansion into a full-fledged e-commerce and repair network. With further enhancements, improved datasets, and expanded features, ElectroFix can revolutionize how users buy, sell, and repair electronic devices.

Appendices

Appendix A: Software Requirements Specification (SRS)

Document Overview

This appendix contains the complete Software Requirements Specification (SRS) for the ElectroFix project. The SRS outlines functional requirements, non-functional requirements, system constraints, system environment, user characteristics, and overall system behavior.

A.1 Introduction

A.1.1 Purpose

The purpose of this SRS document is to formally define all system requirements for the ElectroFix platform. It ensures a clear understanding of system functionalities among developers, clients, and stakeholders.

A.1.2 Scope

The scope includes user registration, authentication, device listing, AI-based pricing, repair booking, order management, payment integration, analytics, and admin control panel.

A.1.3 Definitions, Acronyms, and Abbreviations

- **SRS** — Software Requirements Specification
- **AI** — Artificial Intelligence
- **CRUD** — Create, Read, Update, Delete
- **UI** — User Interface
- **REST API** — Representational State Transfer Application Programming Interface

A.2 Overall Description

A.2.1 Product Perspective

ElectroFix is an integrated platform combining device resale, repair service, and smart AI-based evaluation. The system connects buyers, sellers, and technicians in one central application.

A.2.2 User Classes and Characteristics

- **Customer** — Buys or sells used devices, books repairs.
- **Technician** — Provides repair services.
- **Administrator** — Manages users, devices, transactions.

A.3 Functional Requirements

Below is a structured reproduction of the requirement categories found in your SRS.

A.3.1 User Registration and Login

- Users must be able to register using email and phone number.

- Password validation must meet complexity rules.
- Login must support JWT-based authentication.

A.3.2 Device Buy/Sell Module

- Users can list used devices with images, condition, and specification.
- AI model predicts best price for used device.
- Buyers can view, search, and filter available devices.

A.3.3 Repair Booking System

- Customers can submit repair requests.
- Technicians can accept or reject requests.
- Order tracking available for both customer and technician.

A.4 Non-Functional Requirements

A.4.1 Performance

- The system must support concurrent users without downtime.

A.4.2 Security

- All password data must be stored using hashing algorithms.
- Data exchanged must be secured via HTTPS.

A.4.3 Usability

- The interface must be easy to navigate.

A.5 System Models

“‘latex

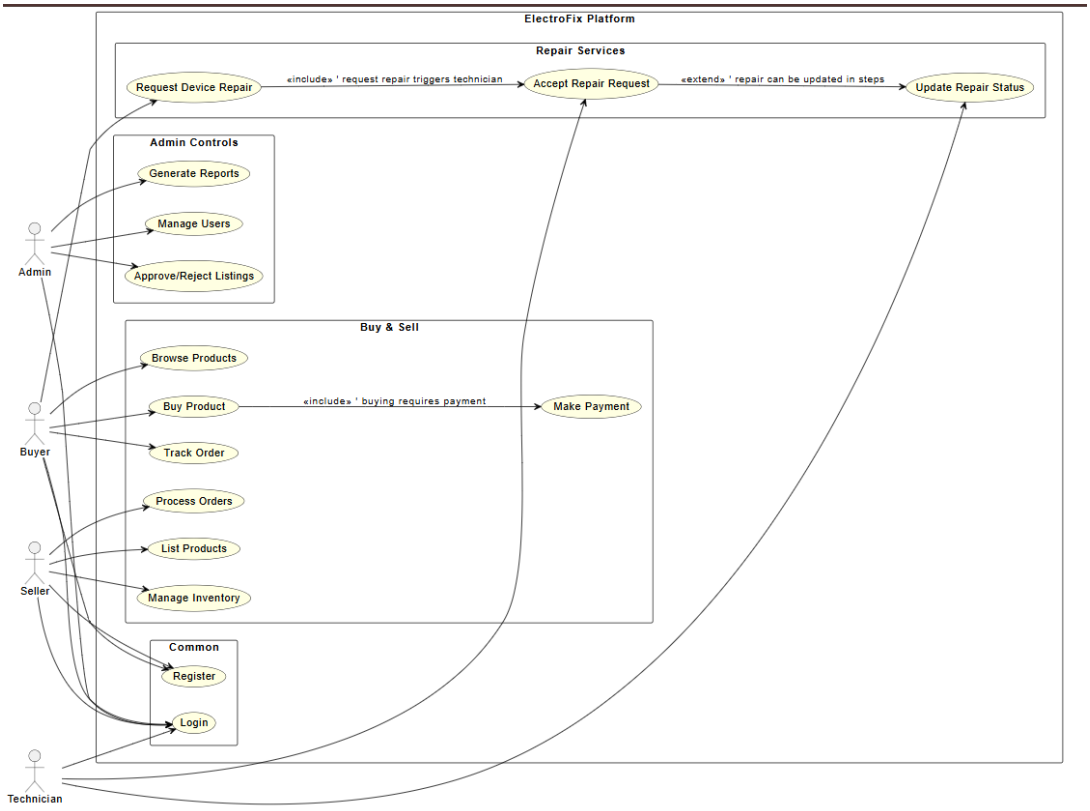


Figure 1: SRS Use Case Diagram

A.6 Software Design Specification (SDS)

This appendix provides a consolidated reference for the Software Design Specification (SDS) of the ElectroFix platform. Since all design elements have already been fully integrated and elaborated within **Chapter 4 – System Design**, this appendix summarizes the key SDS components rather than attaching a separate SDS document.

Summary of SDS Contents

The system design presented in Chapter 4 includes all major components expected in a formal SDS, such as:

- **System Architecture** Layered architecture, deployment view, and interaction among major subsystems.
- **Component-Level Design** Backend modules, frontend modules, API handlers, service layers, and database components.
- **UML Diagrams** Use Case Diagram, Class Diagram, Sequence Diagram, Activity Diagram.
- **Data Flow Diagrams** Level 0 Context Diagram.
- **Database Design** ER Diagram, data schema description, and entity relationships.
- **Security Design** Authentication model, access control logic, data protection, and secure communication flow.

Note to Reader

All SDS elements have been directly incorporated into the report to maintain clarity and avoid redundancy.

All system design specifications are fully covered inside Chapter 4.

A.7 Work Breakdown Structure (WBS)

This appendix contains the Work Breakdown Structure (WBS) used for project planning.

A.8 Work Breakdown Structure (WBS)

This appendix presents the Work Breakdown Structure (WBS) for the ElectroFix project. The WBS outlines the hierarchical decomposition of project tasks and phases, while the corresponding Gantt chart visualizes the project schedule, task durations, and dependencies.

WBS Structure Overview

Structure of WBS

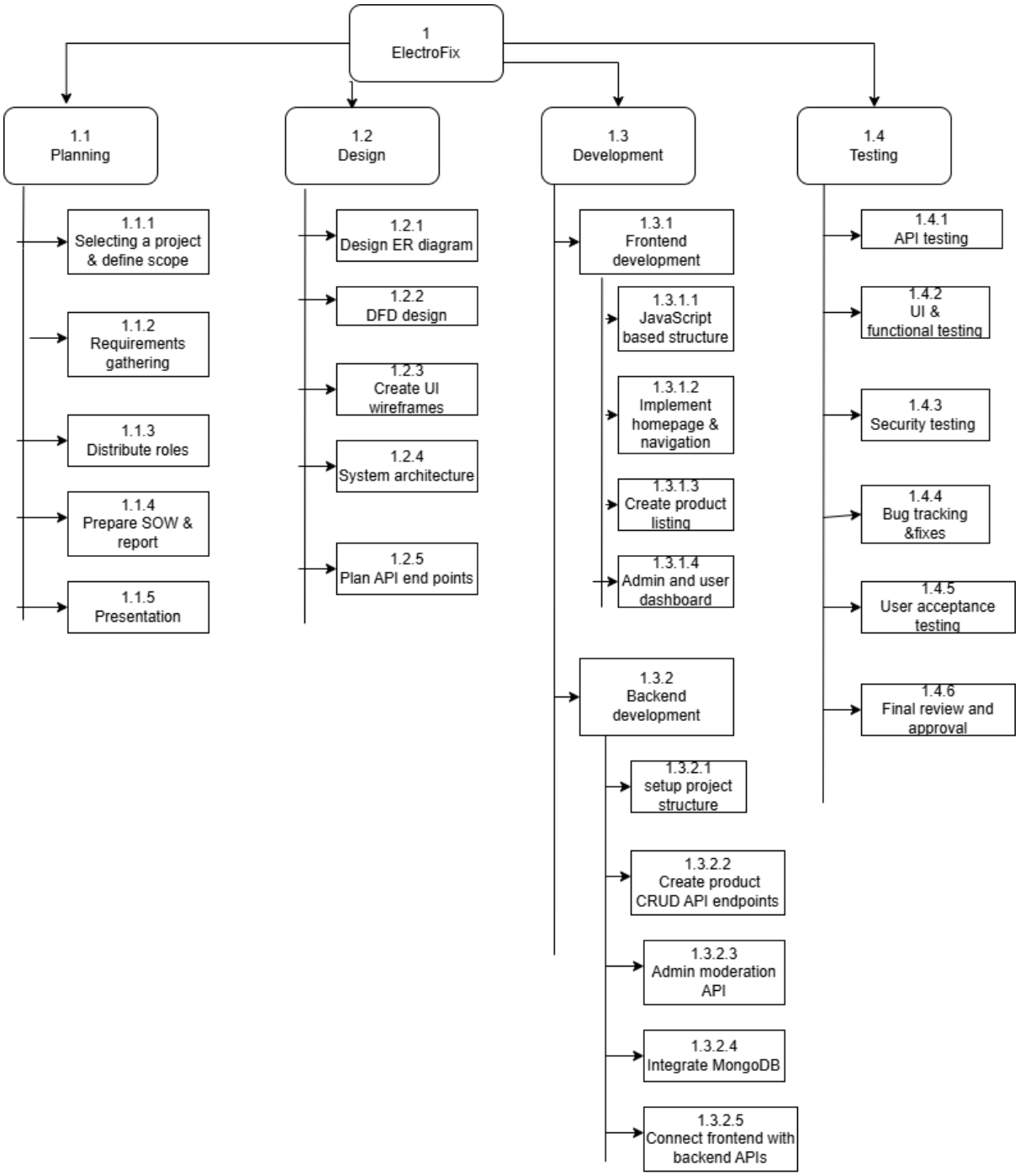


figure: WBS of ElectroFix

Figure 2: ElectroFix Work Breakdown Structure

- **Level-1:** ElectroFix System (entire project scope)
- **Level-2:** Major Phases
  - Requirements Analysis
  - System Design
  - Development
  - Testing
  - Deployment & Delivery
- **Level-3:** Detailed Subtasks under each phase (e.g., UI design, backend modules, database setup, unit testing, integration testing, documentation)

Gantt Chart for WBS

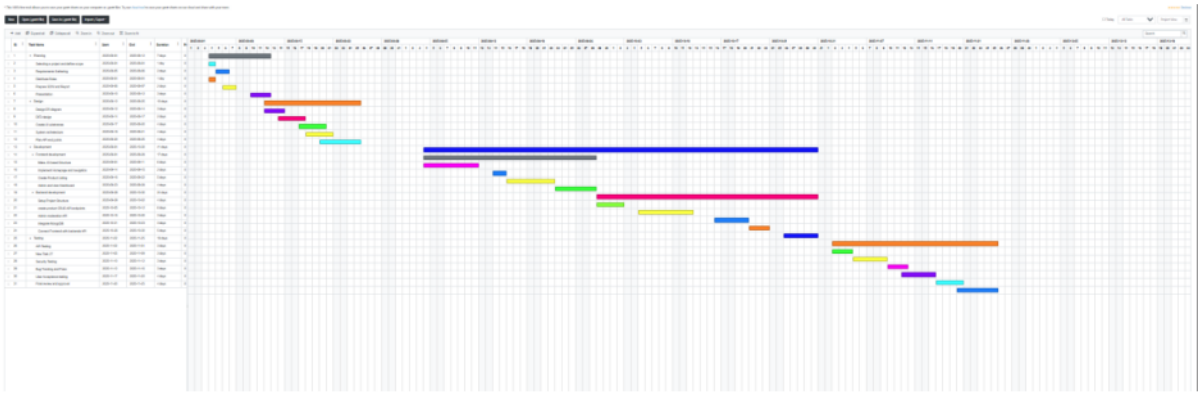


Figure 3: Gantt Chart for ElectroFix Work Breakdown Structure

The full project timeline is represented using a Gantt chart, which includes:

- Duration of each major phase
- Start and end dates
- Task dependencies and sequencing
- Milestones (Requirement Freeze, Beta Release, Final Deployment)
- Resource allocation

The Gantt chart visualizing the Work Breakdown Structure (WBS) is provided on the following page.

A.9 Test Case Template

This appendix presents the Test Case Template used throughout the testing phase of the ElectroFix platform. The template follows the same column structure and formatting used in Chapter 6, ensuring consistency for all recorded test cases such as UI/UX testing, frontend tests, backend API tests, database tests, security tests, performance tests, AI model tests, and integration tests.

Template Format (Aligned With Chapter 6 Tables)

The following template represents the exact format used across all test categories:

Test ID	**Module / Page**	**Test Description**	**Input / Test Data**	**Expected Output**	**Stat**
(Example)	*Login Page*	*Verify login with valid credentials*	*Valid email + password*	*User redirected to dashboard*	*Pass/*

This template is intentionally designed to be simple, readable, and uniform across all test categories. It mirrors the structure used in:

- UI/UX Testing Table
- Frontend Testing Table
- Backend API Testing Table
- Backend Logic Testing Table
- Database Testing Table
- Security Testing Table
- Performance Testing Table
- AI Model Testing Table
- Integration Testing Table

Column Definitions

- **Test ID** Unique identifier such as UX001, F005, A003, DB02, SEC04, PT06, AI-P3, etc.
- **Module / Page** Specific module being tested (Login, Homepage, Payment, API, Database, AI Engine, Admin Panel, etc.)
- **Test Description** Short explanation of what is being tested.
- **Input / Test Data** Values, user actions, or API data used during the test.
- **Expected Output** Correct expected behavior or result.
- **Status** Final result: Pass / Fail.

A.10 Coding Standards Document

This appendix contains the complete coding standards applied throughout ElectroFix development.

Areas Covered

- Naming conventions
- Indentation
- Error handling rules
- API structuring rules
- Python coding conventions (PEP8)
- Folder structure standards
- Version control conventions

A.11 Database Schema and Collections

This appendix includes a detailed database structure for MongoDB.

Collections Used

- **users**
- **products**
- **repairs**
- **predictions**

Sample JSON Documents

User Document

```
{
  "_id": "6729abdf2024",
  "name": "John Doe",
  "email": "john@example.com",
  "password": "hashed_value",
  "role": "customer"
}
```

Product Document

```
{
  "_id": "893a912b",
  "title": "iPhone 11",
  "condition": "Good",
  "expected_price": 22000,
  "age": 2
}
```

Repair Request Document

```
{
  "user_id": "6729abdf2024",
  "device": "Samsung S20",
  "issue": "Does not start",
  "status": "Pending"
}
```

A.12 AI Model Training Dataset Samples

This appendix includes the data samples used to train:

- Price Prediction Model
- Fault Detection CNN Model
- Text-based Fault Classification Model

Dataset Fields (Price Prediction)

- Device Age (Years)
- RAM (GB)
- Storage (GB)
- Physical Condition Score
- Original Market Price

Dataset Fields (Fault Detection)

- Image: Device front/back photos
- Label: Cracked Screen / Liquid Damage / Battery Issue / Others

Sample Dataset Table

Age	RAM	Storage	Market Price
1.5	6GB	128GB	48000
2.0	4GB	64GB	29000
3.0	6GB	256GB	22000

A.13 System Screenshots

Screenshots of the ElectroFix system are included here for visual demonstration of the final UI and workflow.

Suggested Screenshots (Insert Images)

- Homepage
- Login Page
- Signup page
- Sell Device Form
- Repair Booking Page
- Dashboard

A.14 Application Screenshots

This section includes the key interface pages of the ElectroFix platform.

Homepage

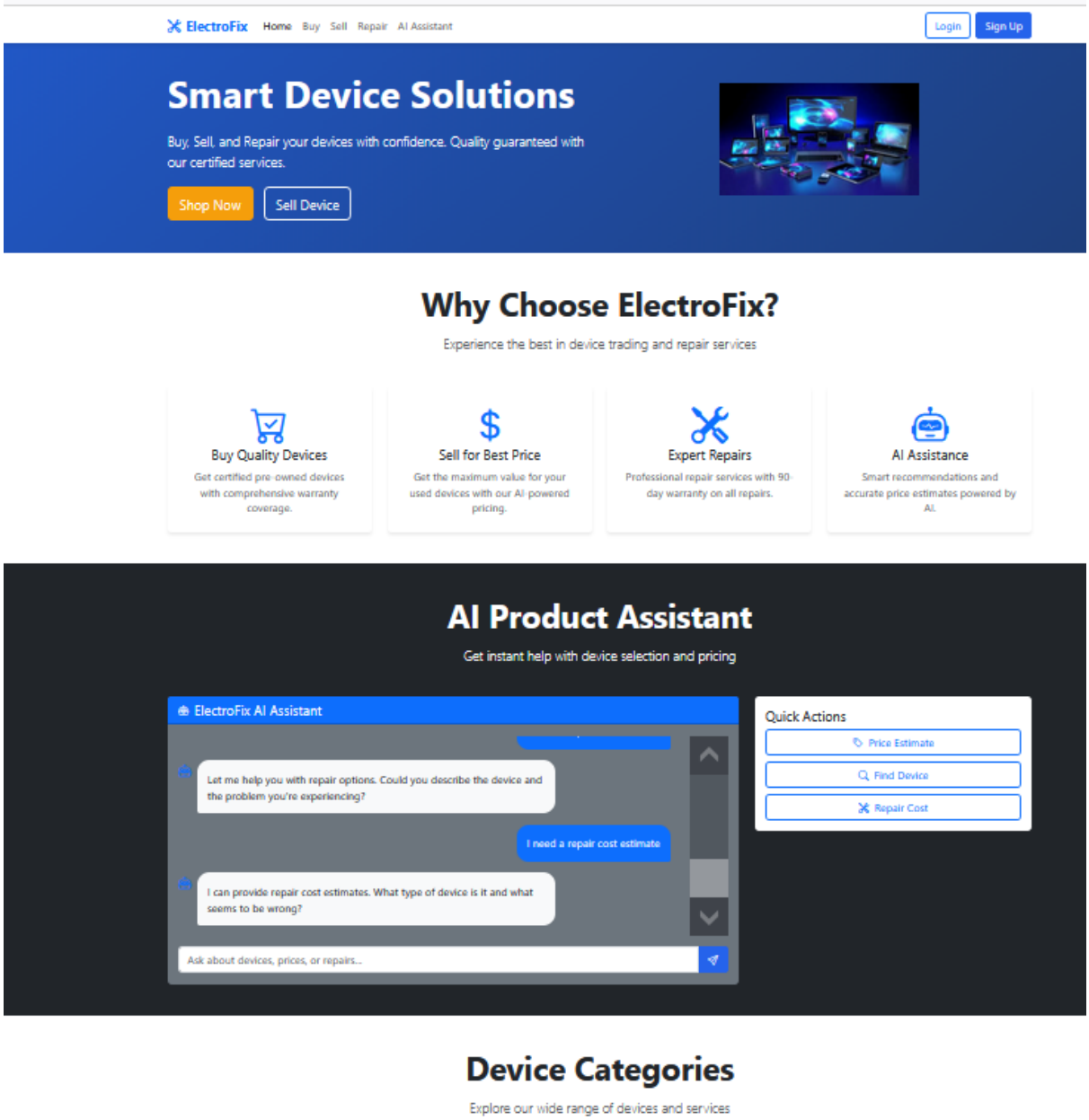


Figure 4: ElectroFix Homepage Interface

Login Page

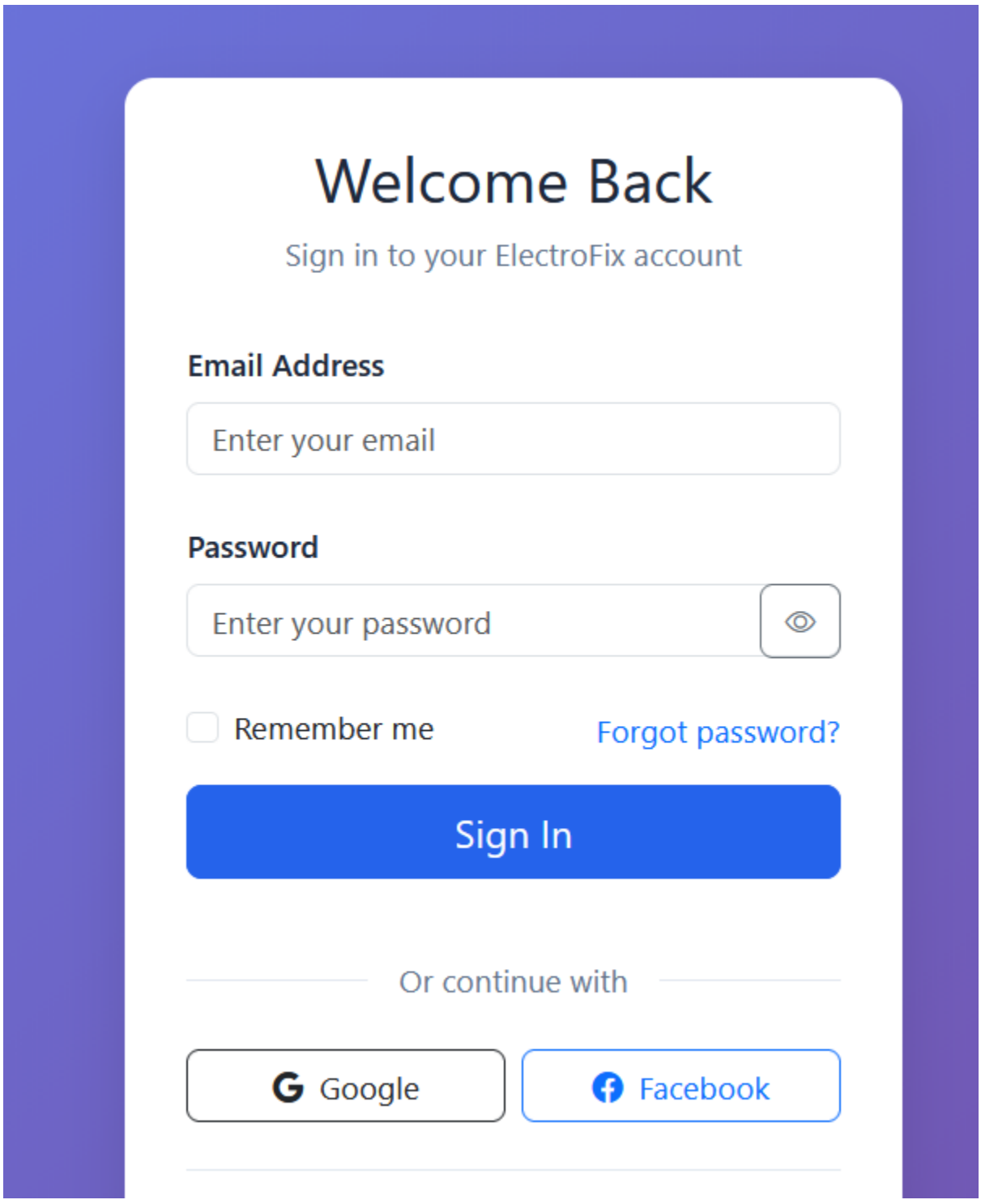
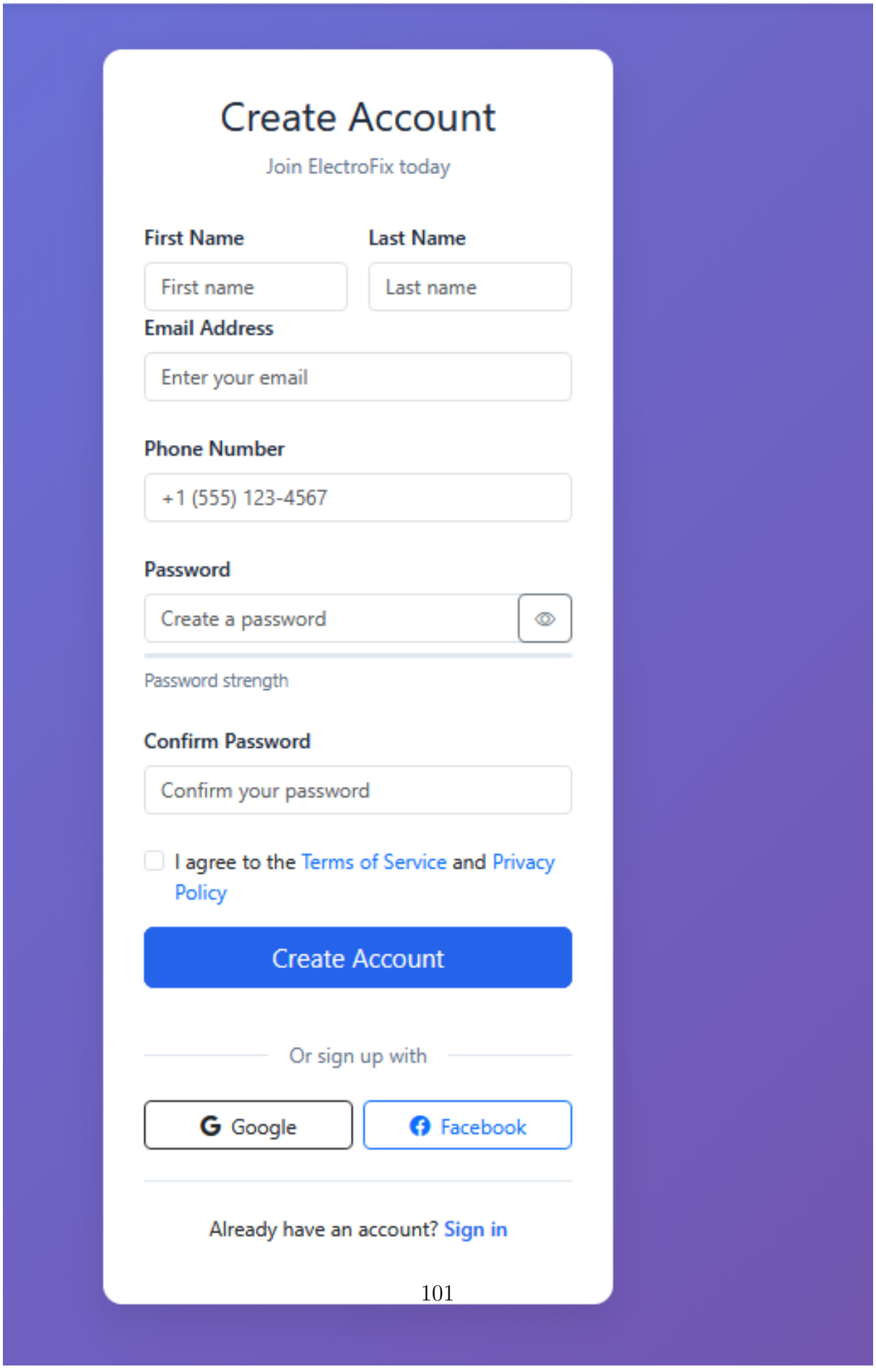


Figure 5: Login Page Interface

Signup Page


Create Account
Join ElectroFix today
First Name
Last Name
First name
Last name
Email Address
Enter your email
Phone Number
+1 (555) 123-4567
Password
Create a password
Password strength
Confirm Password
Confirm your password
I agree to the Terms of Service and Privacy Policy
Create Account
Or sign up with
Google
Facebook
Already have an account? Sign in

Dashboard Page

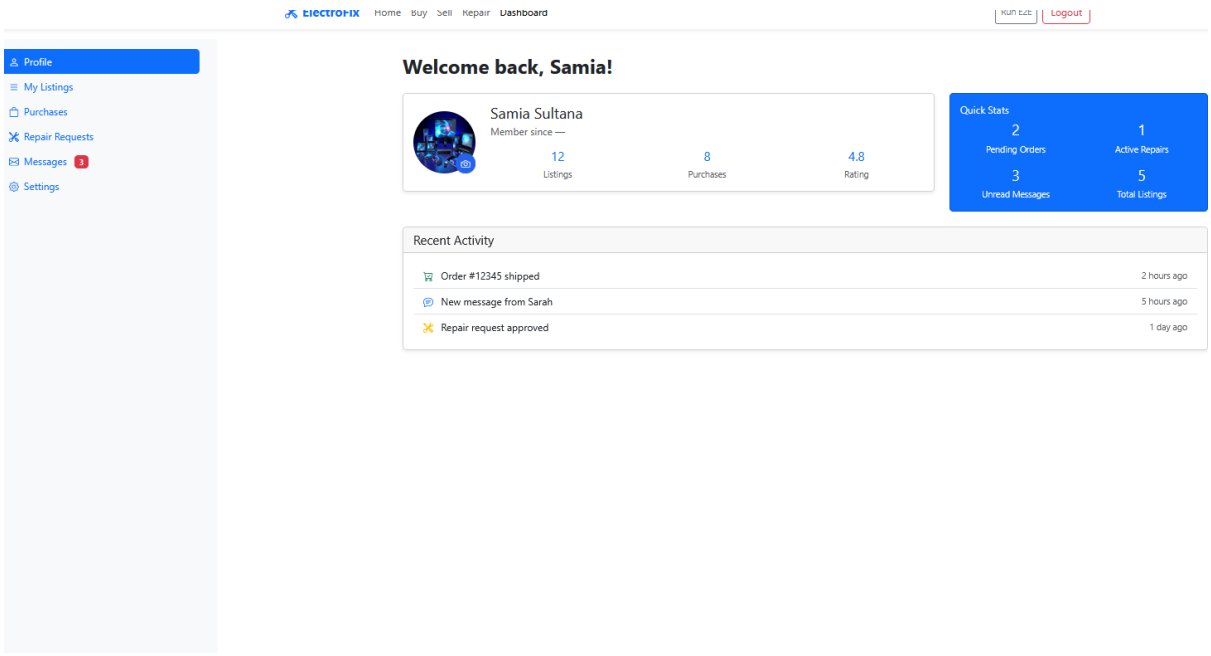


Figure 7: User Dashboard Interface

Sell Device Form

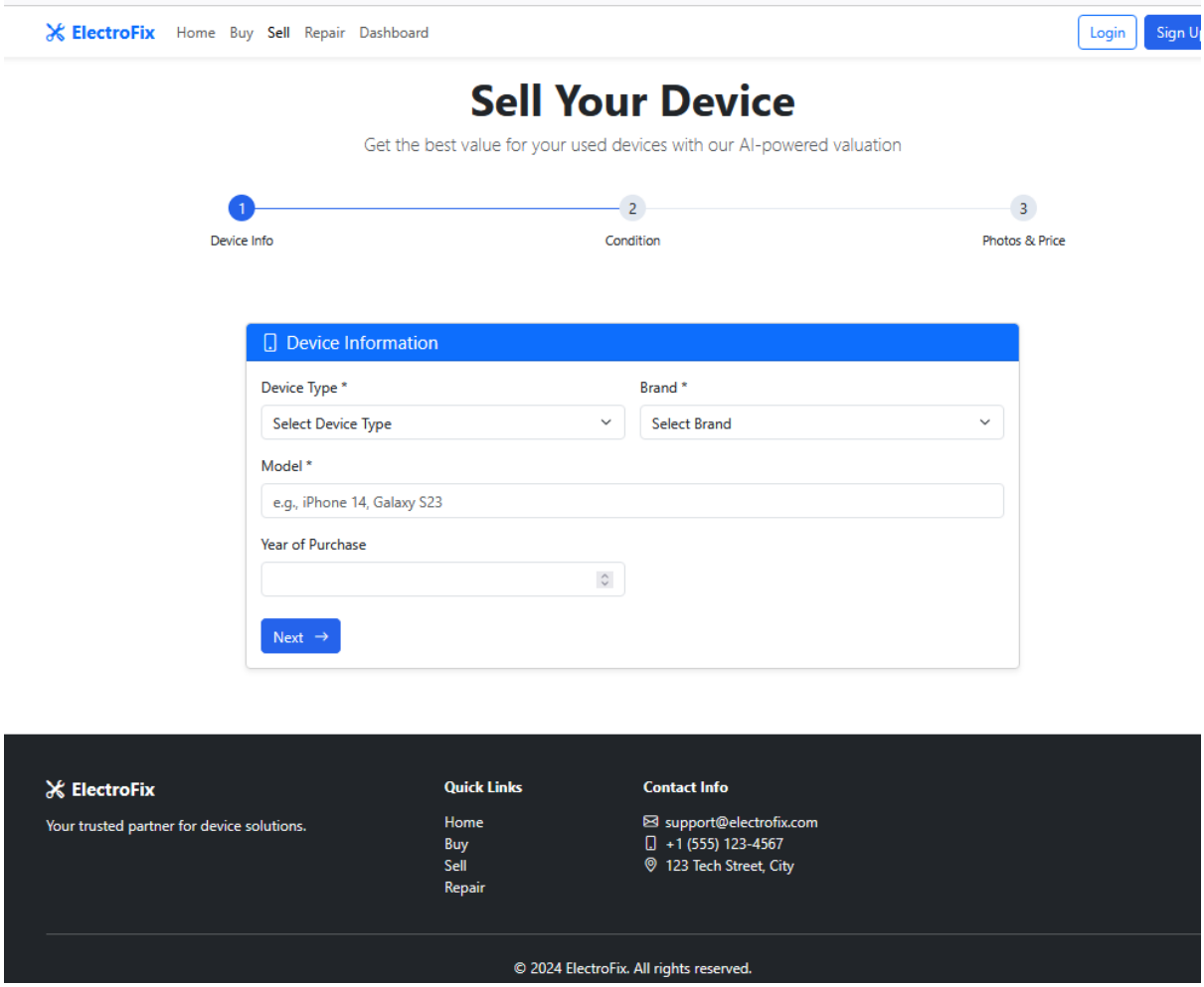


Figure 8: Sell Device Form Interface

Repair Booking Page

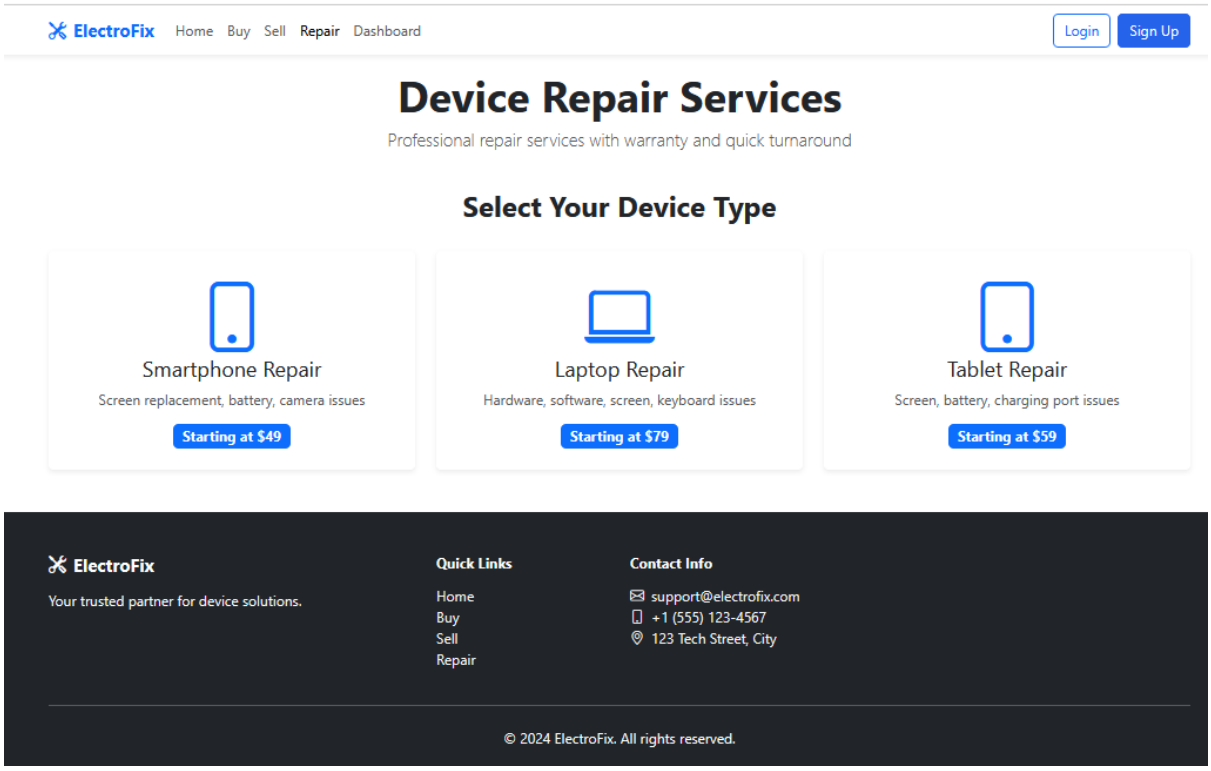


Figure 9: Repair Booking Page Interface

Repair Booking Form

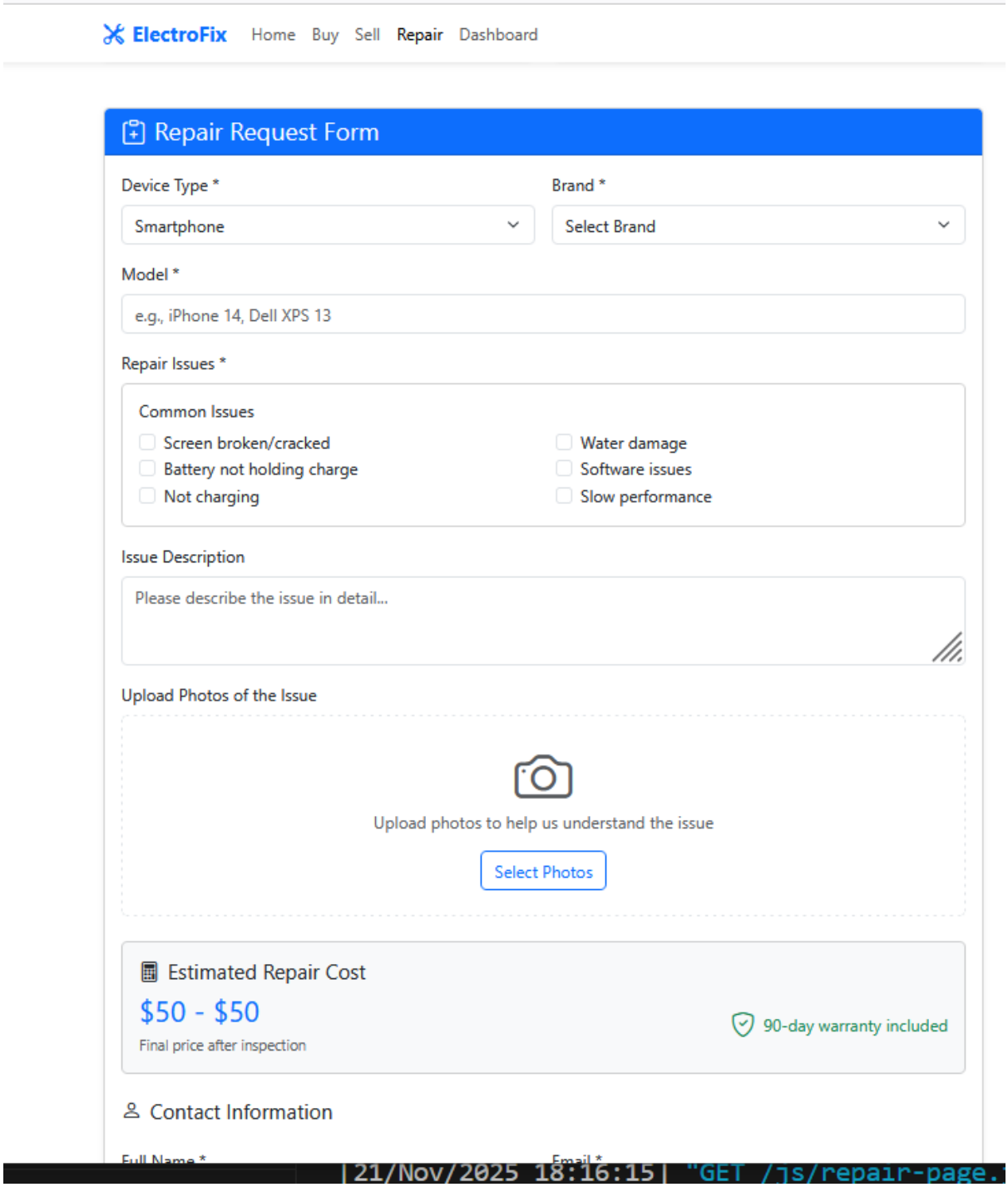


Figure 10: Repair Booking Form Interface

AI Bot

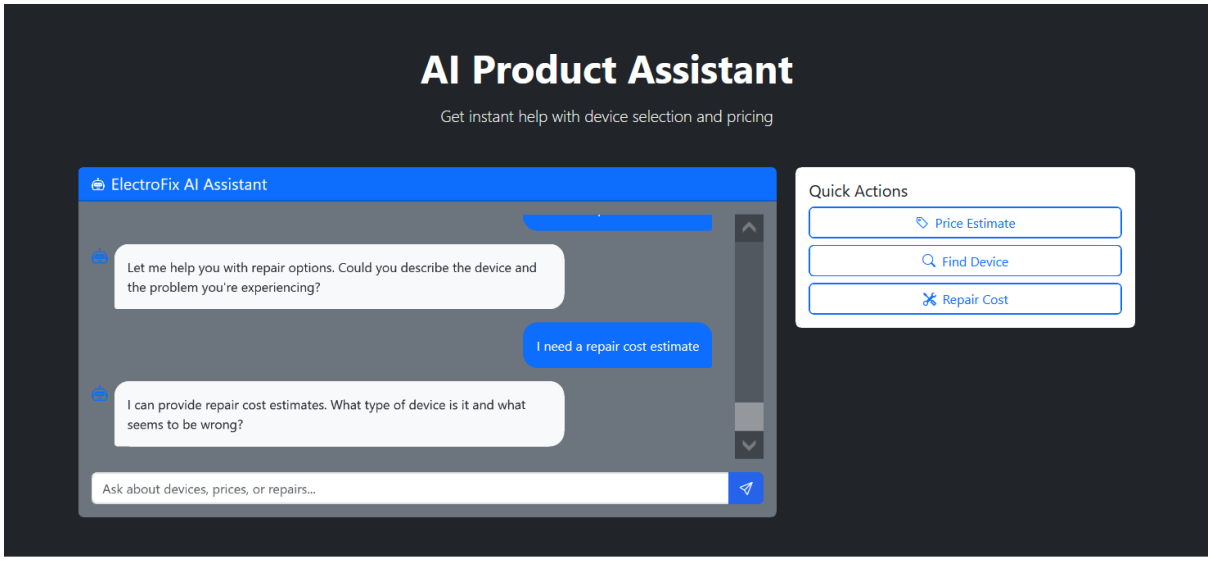


Figure 11: AI chat Bot Interface

Bibliography

[1] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2015.

[2] R. S. Pressman and B. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill, 2014.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[4] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. Cham, Switzerland: Springer, 2020.

[5] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly Media, 2023.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[7] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.

[8] W. Stallings, *Cryptography and Network Security*, 7th ed. Pearson, 2016.

[9] "Ieee std 830-1998: Recommended practice for software requirements specifications," Tech. Rep., 1998.

[10] "Ieee std 1016-2009: Standard for software design descriptions," Tech. Rep., 2009.

[11] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Addison-Wesley, 2003.

[12] "Aws cloud architecture documentation," https://aws.amazon.com/architecture/, Amazon Web Services, 2024.

[13] "Mongodb documentation," https://docs.mongodb.com/, MongoDB Inc., 2024.

[14] "Flask documentation," https://flask.palletsprojects.com/, Pallets Projects, 2024.

[15] "Json web tokens (jwt) documentation," https://jwt.io/introduction, Auth0 Inc., 2024.

[16] "scikit-learn documentation," https://scikit-learn.org/, Scikit-learn Developers, 2024.

[17] "Tensorflow documentation," https://www.tensorflow.org/, TensorFlow Developers, 2024.

[18] "Pytorch documentation," https://pytorch.org/, PyTorch Team, 2024.

[19] "Ieee std 829-2008: Standard for software test documentation," Tech. Rep., 2008.

[20] "Postman api testing tools," https://www.postman.com/, Postman, 2024.

[21] "Jmeter documentation," https://jmeter.apache.org/, Apache Foundation, 2024.

[22] M. Xu, K. Zhao, and H. Jin, "Ai-driven price prediction for used electronic devices," *IEEE Access*, vol. 8, pp. 145 200–145 210, 2020.

[23] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.

[24] E. Marcotte, "Responsive web design," *A List Apart*, 2010.

[25] "Bootstrap 5 documentation," https://getbootstrap.com/, Bootstrap Team, 2024.